2025-12-01

Automotive Artificial Intellegence

2D Semantic Segmentation – 1조

자동차공학과 20243299 박시현
미래모빌리티학과 20243422 오효정
미래모빌리티학과 20233391 김채연

Content

01 2D Sematic Segmentation

02 Model Introduction
FCN, Unet

03 EDA

04 Model Selection Process

05 Final Model Selection

06 Limitations and Improvement Measures

1. 2D Semantic Segmentation

CONTENTS

01 2D Sematic Segmentation

02 Model Introduction
FCN, Unet

03 EDA

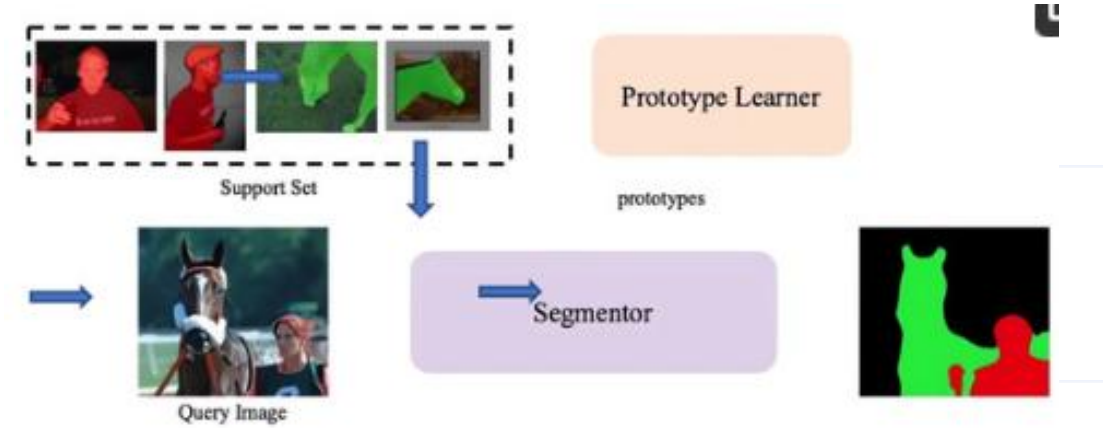
04 Model Selection Process

05 Final Model Selection

06 Limitations and Improvement Measures

2D Sematic Segmentation

- Not distinguish the exact boundaries of each object
- Classify objects by marking all pixels belonging to the same object type with the same label



▲ https://www.mdpi.com/1999-5903/15/6/205



Input

Semantic Labels

▲ https://www.jeremyjordan.me/semantic-segmentation/

2. Model Introduction

CONTENTS

01 2D Sematic Segmentation

02 Model Introduction
FCN, Unet

03 EDA

04 Model Selection Process

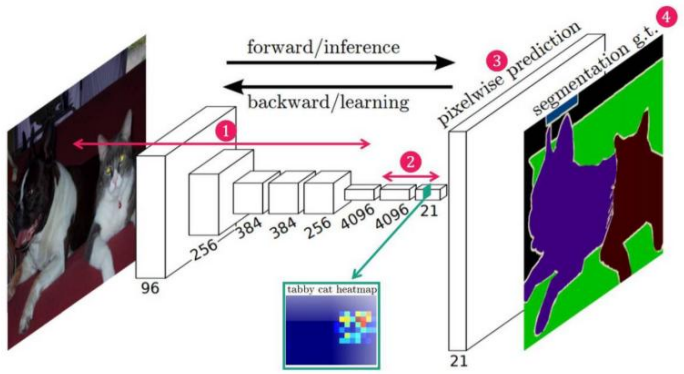
05 Final Model Selection

06 Limitations and Improvement Measures

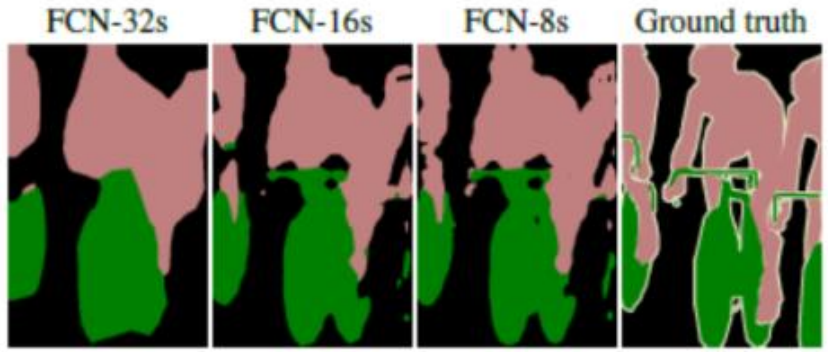
FCN (Fully Convolutional Network)

Due to the fully connected the model loses spatial information from the original image and requi a fixed input image size

- 1X1 Convolution Layer
- Down Sampling(Encoding)
- Up Sampling allows the network recover the original input image size
  - Skip connection Architecture



https://wikidocs.net/147359



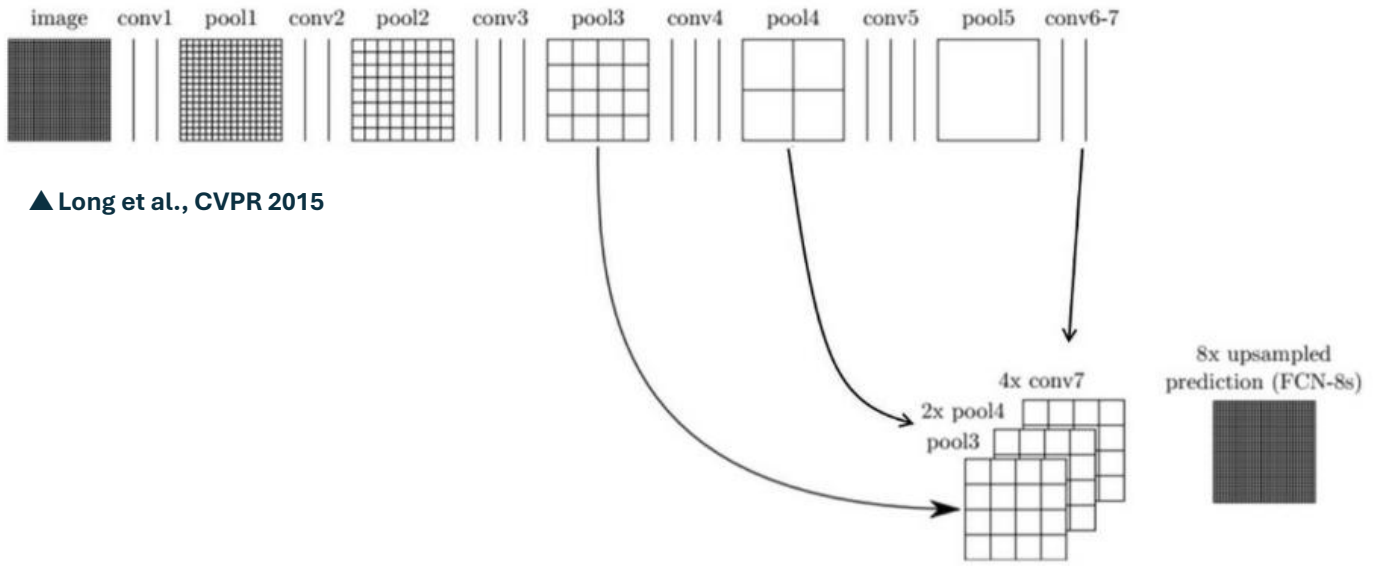
▲ Long et al., CVPR 2015

FCN (Fully Convolutional Network)

Up Sampling

- Stride Transposed Convolution
- Skip connection Architecture: Expand the current Feature Map and add the information from the previous Feature Map

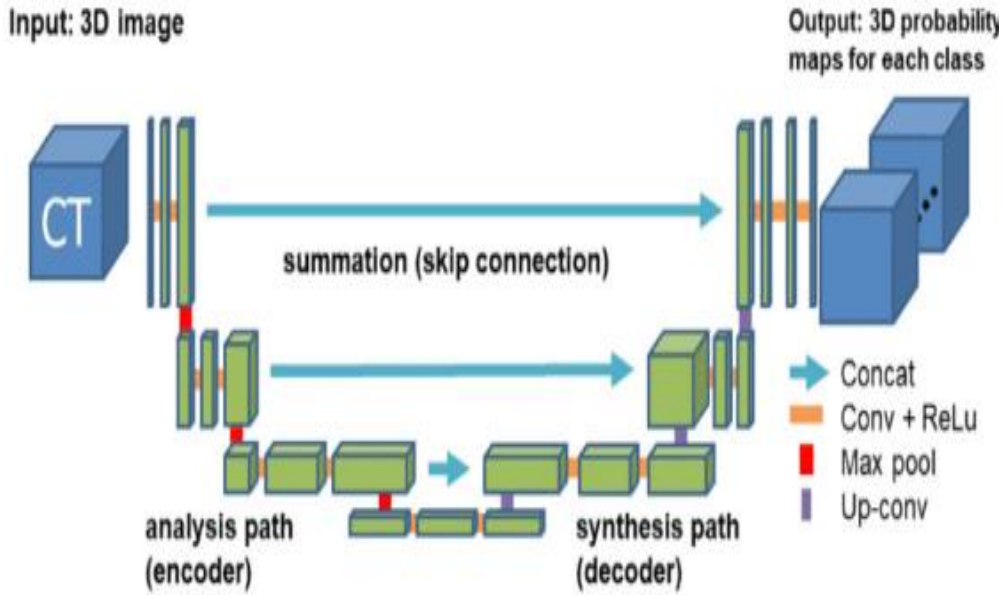
➡
  - FCN-32s
  - FCN-16s
  - FCN-8s

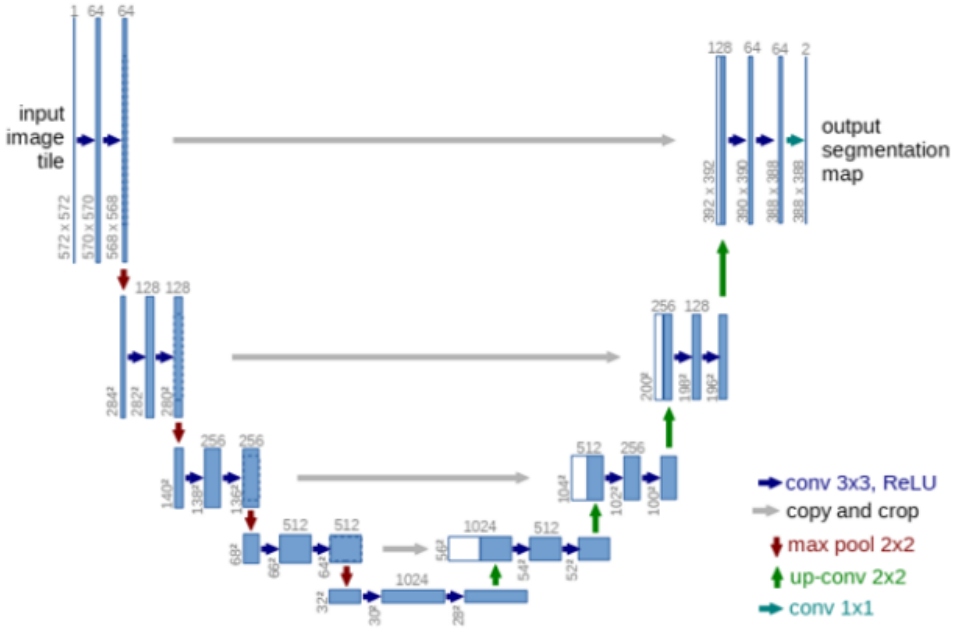


▲ Long et al., CVPR 2015

U-Net

- It has a U-shaped, symmetric architecture composed of an encoder, decoder, and a bridge.
- In the original model, downsampling drops spatial details; U-Net restores them through skip connections.





▲ https://theaisummer.com/unet-architectures/

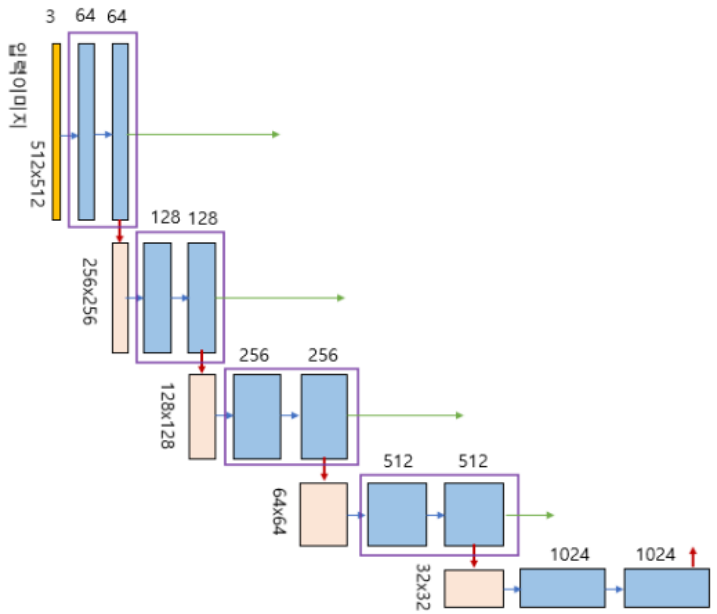
https://pasus.tistory.com/204

U-Net

U- Net Architecture

- Contracting Path(Encoder)

: It reduces spatial resolution while increasing channels to capture richer features from the input image



- 3×3 convolution, batch normalization, and ReLU in order.
- Maxpooling
- Copying feature maps for skip connections

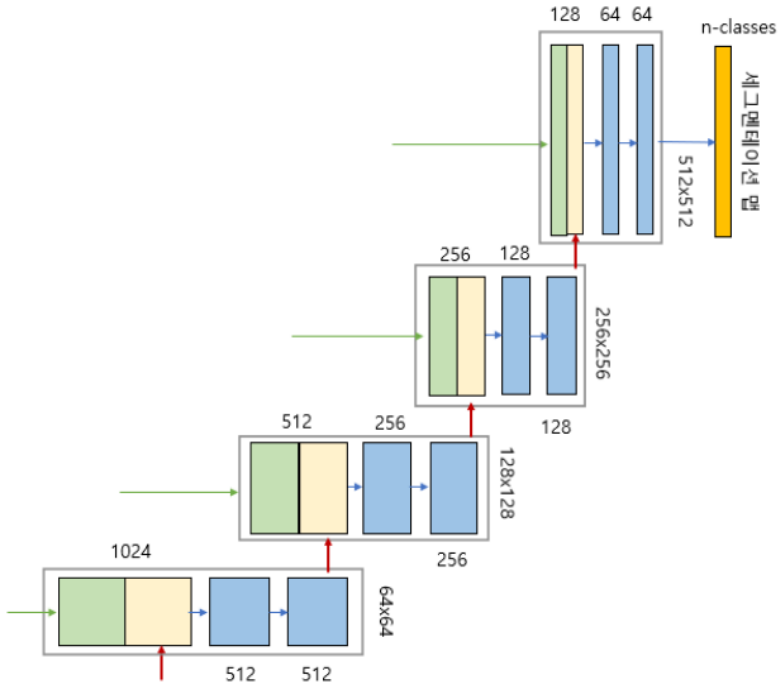
https://pasus.tistory.com/204

U-Net

U- Net Architecture

- Expending Path(Decoder)

: Reconstructing the image using only the low level encoded features



- 3×3 convolution, batch normalization, and ReLU in order
- Transposed convolution
- Skip connection

https://pasus.tistory.com/204

3. EDA

CONTENTS

01 2D Sematic Segmentation

02 Model Introduction
FCN, Unet

03 EDA

04 Model Selection Process

05 Final Model Selection

06 Limitations and Improvement Measures

EDA

- **2D Semantic Segmentation for Self Driving Cars**

  - Dataset with Semantic Segmentation Labels generated via CARLA simulator

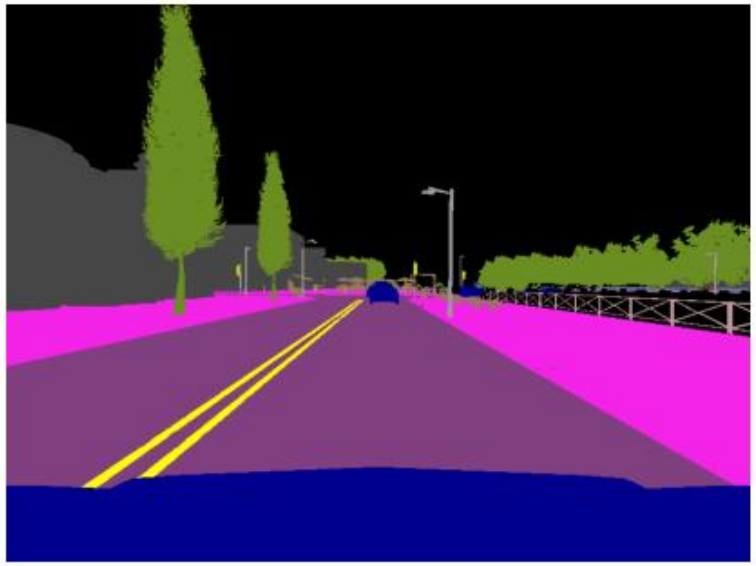
  - https://www.kaggle.com/datasets/kumaresanmanickavelu/lyft-udacity-challenge

  - Training 1000 images, validation 1000 images (no test set)

RGB (F68-62.png)



Color Label



Value	Tag
0	None
1	Buildings
2	Fences
3	Other
4	Pedestrians
5	Poles
6	RoldLines
7	Roads
8	Sidewalks
9	Vegetation
10	Vehicles
11	Walks
12	TrafficSigns

EDA

- **Dataset Description**

  - All images have a **uniform size** of **800x600**

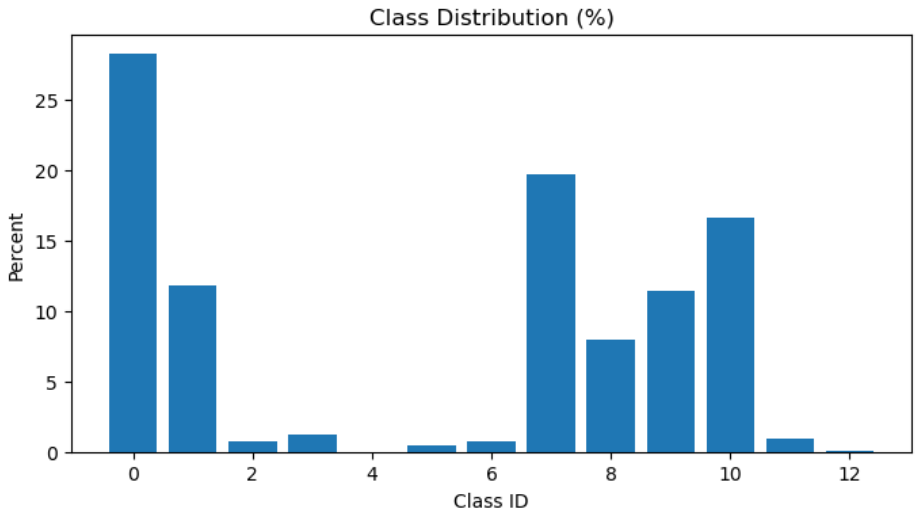
  - Most pixels belong to Roads, Builds, Vegetation

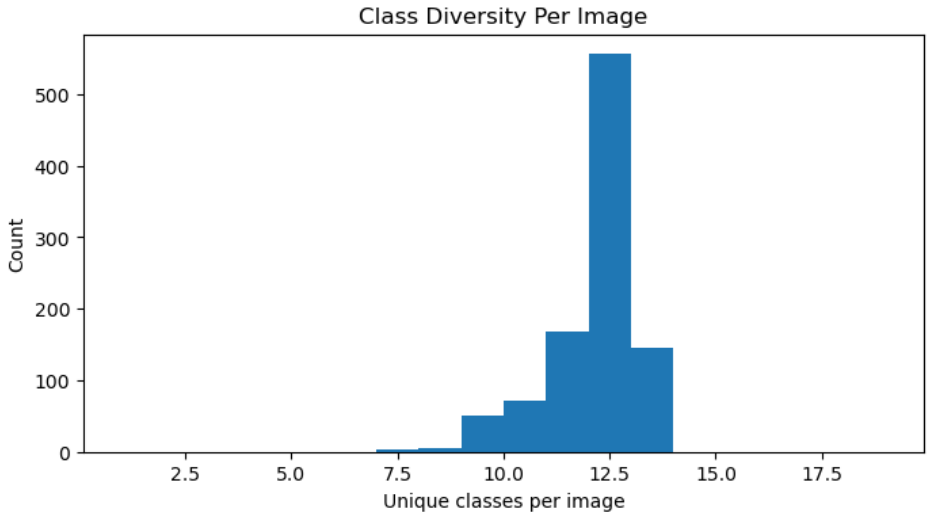
  - Pedestrians / RoadLines / Traffic Signs extremely rare

  - Model can be biased -> must handle **imbalance**

```
Class pixel ratio (%):
Class 0: 28.237%
Class 1: 11.804%
Class 2: 0.781%
Class 3: 1.202%
Class 4: 0.006%
Class 5: 0.490%
Class 6: 0.744%
Class 7: 19.667%
Class 8: 7.973%
Class 9: 11.463%
Class 10: 16.661%
Class 11: 0.893%
Class 12: 0.081%
```

```
total images: 1000
class 0: appears in 1000 images (100.0%)
class 1: appears in 900 images (90.0%)
class 2: appears in 994 images (99.4%)
class 3: appears in 884 images (88.4%)
class 4: appears in 180 images (18.0%)
class 5: appears in 991 images (99.1%)
class 6: appears in 972 images (97.2%)
class 7: appears in 1000 images (100.0%)
class 8: appears in 1000 images (100.0%)
class 9: appears in 1000 images (100.0%)
class 10: appears in 1000 images (100.0%)
class 11: appears in 777 images (77.7%)
class 12: appears in 946 images (94.6%)
```





4. Model Selection Process

CONTENTS

01 2D Sematic Segmentation

02 Model Introduction
FCN, Unet

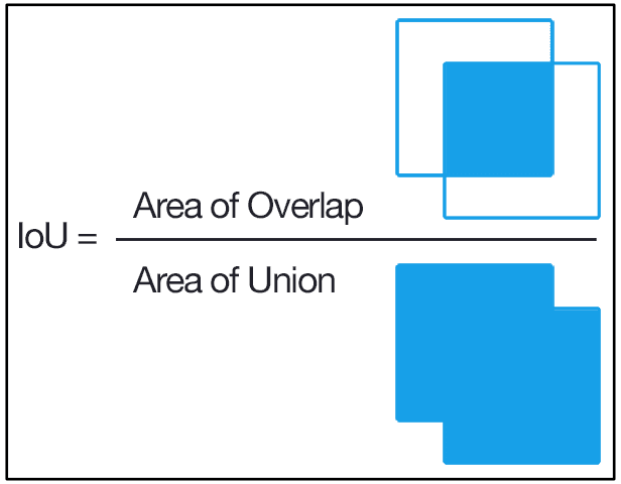
03 EDA

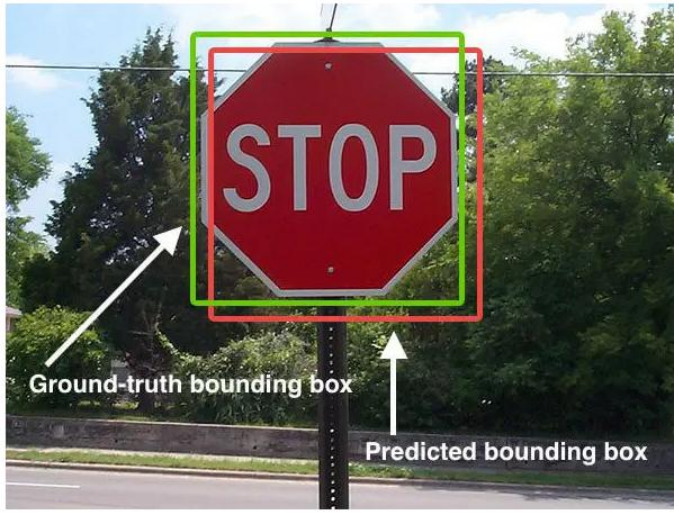
04 Model Selection Process

05 Final Model Selection

06 Limitations and Improvement Measures

IoU(Intersection over Unit)





MIoU: Mean of IoU

▲ https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/

- An evaluation metric used to measure the accuracy of an object detector on a particular dataset
- Intersection / Union

K-FCNConcat(Early Model)

```python
class khaha(nn.Module):
    def __init__(self, num_classes=13):
        super().__init__()
        self.leaky = nn.LeakyReLU(0.1)

        # === Encoder ===
        self.enc1_1 = nn.Conv2d(3, 64, 3, padding=1)
        self.enc1_2 = nn.Conv2d(64, 64, 3, padding=1)
        self.pool1 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

        self.enc2_1 = nn.Conv2d(64, 128, 3, padding=1)
        self.enc2_2 = nn.Conv2d(128, 128, 3, padding=1)
        self.pool2 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

        self.enc3_1 = nn.Conv2d(128, 256, 3, padding=1)
        self.enc3_2 = nn.Conv2d(256, 256, 3, padding=1)
        self.pool3 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

        self.enc4_1 = nn.Conv2d(256, 512, 3, padding=1)
        self.enc4_2 = nn.Conv2d(512, 512, 3, padding=1)
        self.pool4 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

        self.enc5_1 = nn.Conv2d(512, 1024, 3, padding=1)
        self.enc5_2 = nn.Conv2d(1024, 1024, 3, padding=1)
        self.pool5 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

        self.enc6_1 = nn.Conv2d(1024, 1024, 3, padding=1)
        self.enc6_2 = nn.Conv2d(1024, 1024, 3, padding=1)
```

```python
def forward(self, x):
    # --- Encoding ---
    x = self.leaky(self.enc1_1(x))
    x = self.leaky(self.enc1_2(x))
    skip1 = x
    x = self.pool1(x)

    x = self.leaky(self.enc2_1(x))
    x = self.leaky(self.enc2_2(x))
    skip2 = x
    x = self.pool2(x)

    x = self.leaky(self.enc3_1(x))
    x = self.leaky(self.enc3_2(x))
    skip3 = x
    x = self.pool3(x)

    x = self.leaky(self.enc4_1(x))
    x = self.leaky(self.enc4_2(x))
    skip4 = x
    x = self.pool4(x)
```

```python
# 5. Training Loop
############################################################
if 'train_loader' in locals():
    print("학습을 시작합니다... (잠시만 기다려주세요)")
    NUM_EPOCHS = 20
    for epoch in range(NUM_EPOCHS):
        model.train()
        total_loss = 0

        for img, label in train_loader:
            img, label = img.to(device), label.to(device)

            output = model(img)
            loss = criterion(output, label)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

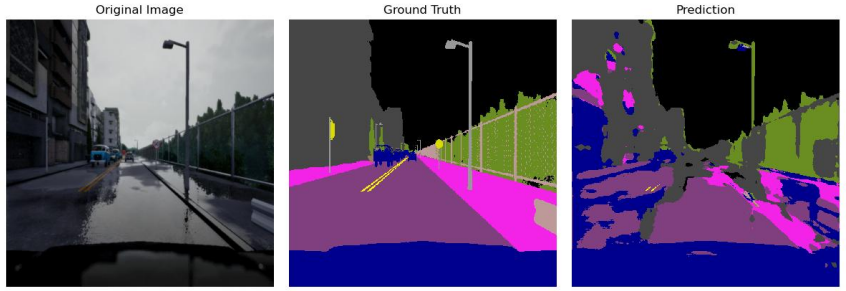
        avg_loss = total_loss / len(train_loader)
        print(f"[Epoch {epoch+1}/{NUM_EPOCHS}] Train Loss: {avg_loss:.4f}")

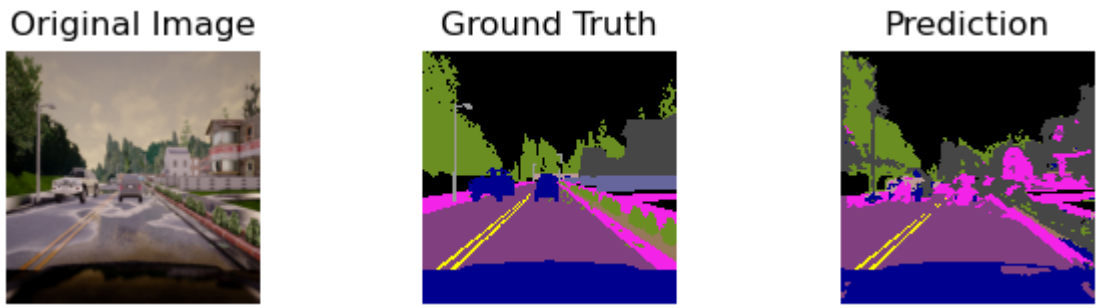
    print("🎉 모든 학습이 완료되었습니다!")
    torch.save(model.state_dict(), "fcn16_multi_class_aug.pth")
```

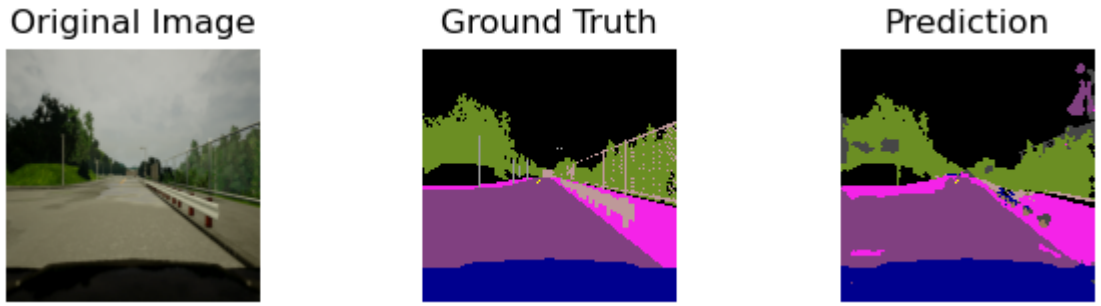
- Downsampled the resolution with 6 Conv Layer, compressed to 1024 channels
- Increased the image resolution using Transposed convolution

- Borrowed U-Net Concatenation to add the output of the encoder to the input of decoder in the decoding layers

K-FCNConcat(Early Model)








```
----------------------------------------------------
  Class-wise IoU:
    Class 0: 0.8547
    Class 1: 0.2830
    Class 2: 0.0257
    Class 3: 0.0436
    Class 4: 0.0000
    Class 5: 0.0000
    Class 6: 0.5214
    Class 7: 0.6111
    Class 8: 0.3100
    Class 9: 0.5464
    Class 10: 0.7080
    Class 11: 0.0000
    Class 12: 0.0000
  mIoU (Mean IoU): 0.3003
====================================================
```

- Scheduler X
- BatchNorm X
- Augmentation X

K-FCNConcat

```
x = self.leaky(self.bn5_1(self.conv5_1(x)))
x = self.leaky(self.bn5_2(self.conv5_2(x)))
skip5 = x
x = self.pool5(x)

x = self.leaky(self.bn6_1(self.conv6(x)))
```

```
self.upconv5 = nn.ConvTranspose2d(1024, 512, 2, stride=2)
self.dec5_1 = nn.Conv2d(1536, 512, 3, padding=1)
self.bn_dec5_1 = nn.BatchNorm2d(512)
self.dec5_2 = nn.Conv2d(512, 512, 3, padding=1)
self.bn_dec5_2 = nn.BatchNorm2d(512)

self.upconv4 = nn.ConvTranspose2d(512, 256, 2, stride=2)
self.dec4_1 = nn.Conv2d(768, 256, 3, padding=1)
self.bn_dec4_1 = nn.BatchNorm2d(256)
self.dec4_2 = nn.Conv2d(256, 256, 3, padding=1)
self.bn_dec4_2 = nn.BatchNorm2d(256)

self.upconv3 = nn.ConvTranspose2d(256, 128, 2, stride=2)
self.dec3_1 = nn.Conv2d(384, 128, 3, padding=1)
self.bn_dec3_1 = nn.BatchNorm2d(128)
self.dec3_2 = nn.Conv2d(128, 128, 3, padding=1)
self.bn_dec3_2 = nn.BatchNorm2d(128)

self.upconv2 = nn.ConvTranspose2d(128, 64, 2, stride=2)
self.dec2_1 = nn.Conv2d(192, 64, 3, padding=1)
self.bn_dec2_1 = nn.BatchNorm2d(64)
self.dec2_2 = nn.Conv2d(64, 64, 3, padding=1)
self.bn_dec2_2 = nn.BatchNorm2d(64)
```

```
for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for images, labels in train_loader:  # train_loader 준비 필요
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * images.size(0)

    scheduler.step()
    print(f"Epoch {epoch+1}/{num_epochs} - Loss: {epoch_loss:.4f}, Current Learning Rate: {current_lr:.6f}")
```

```
if self.mode == 'train':
    # 1. Random Resized Crop (자르고 -> 다시 원래 크기로 복구)
    # scale=(0.5, 1.0): 이미지의 50%~100% 영역 중 랜덤하게 자름
    i, j, h, w = transforms.RandomResizedCrop.get_params(
        img, scale=(0.9, 1.0), ratio=(0.9, 1.1)
    )
    # 이미지: Bilinear (부드럽게), 라벨: Nearest (값 유지)
    img = TF.resized_crop(img, i, j, h, w, size=IMG_SIZE, interpolation=transforms.InterpolationMode.BILINEAR)
    label = TF.resized_crop(label, i, j, h, w, size=IMG_SIZE, interpolation=transforms.InterpolationMode.NEAREST)

    # 2. Random Horizontal Flip (좌우 대칭 50%)
    if random.random() < 0.5:
        img = TF.hflip(img)
        label = TF.hflip(label)
```

- Reduce Conv6 layer to single convolution layer
- Uses BatchNormalization

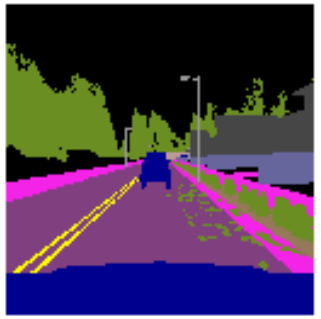
- Uses Augmentation
- Use Scheduler

K-FCNConcat

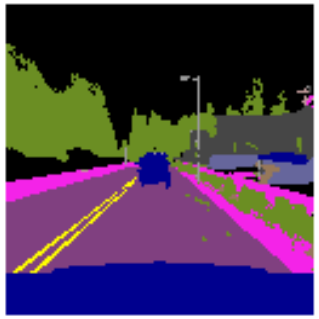
Original Image

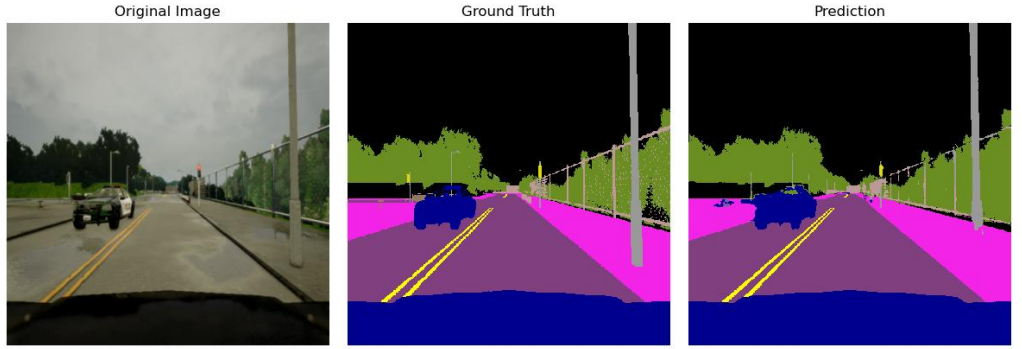


Ground Truth



Prediction





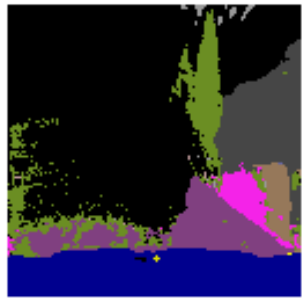
Original Image



Ground Truth



Prediction



```
--------------------------------------------------
  Class-wise IoU:
    Class 0: 0.8645
    Class 1: 0.7958
    Class 2: 0.2419
    Class 3: 0.1210
    Class 4: 0.0000
    Class 5: 0.2914
    Class 6: 0.6923
    Class 7: 0.8325
    Class 8: 0.7269
    Class 9: 0.7165
    Class 10: 0.9570
    Class 11: 0.2147
    Class 12: 0.5105
  mIoU (Mean IoU): 0.5358
==================================================
```

Limitation

```python
def __init__(self, num_classes):
    super(khaha, self).__init__()
    self.leaky = nn.ReLU(0.1)
    self.dropout = nn.Dropout2d(0.3)
    # ---------- Encoder ----------
    self.conv1_1 = nn.Conv2d(3,64,3,padding=1)
    self.bn1_1 = nn.BatchNorm2d(64)
    self.conv1_2 = nn.Conv2d(64,64,3,padding=1)
    self.bn1_2 = nn.BatchNorm2d(64)
    self.pool1 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

    self.conv2_1 = nn.Conv2d(64,128,3,padding=1)
    self.bn2_1 = nn.BatchNorm2d(128)
    self.conv2_2 = nn.Conv2d(128,128,3,padding=1)
    self.bn2_2 = nn.BatchNorm2d(128)
    self.pool2 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

    self.conv3_1 = nn.Conv2d(128,256,3,padding=1)
    self.bn3_1 = nn.BatchNorm2d(256)
    self.conv3_2 = nn.Conv2d(256,256,3,padding=1)
    self.bn3_2 = nn.BatchNorm2d(256)
    self.pool3 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

    self.conv4_1 = nn.Conv2d(256,512,3,padding=1)
    self.bn4_1 = nn.BatchNorm2d(512)
    self.conv4_2 = nn.Conv2d(512,512,3,padding=1)
    self.bn4_2 = nn.BatchNorm2d(512)
    self.pool4 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

    self.conv5_1 = nn.Conv2d(512,1024,3,padding=1)
    self.bn5_1 = nn.BatchNorm2d(1024)
    self.conv5_2 = nn.Conv2d(1024,1024,3,padding=1)
    self.bn5_2 = nn.BatchNorm2d(1024)
    self.pool5 = nn.MaxPool2d(2, stride=2, ceil_mode=True)

    self.conv6 = nn.Conv2d(1024,1024,3,padding=1)
    self.bn6_1 = nn.BatchNorm2d(1024)
    self.conv6_1 = nn.Conv2d(1024,1024,3,padding=1)
    self.bn6_2 = nn.BatchNorm2d(1024)
    self.conv6_2 = nn.Conv2d(1024,1024,3,padding=1)
    self.bn6_3 = nn.BatchNorm2d(1024)
    self.pool6 = nn.MaxPool2d(2, stride=2, ceil_mode=True)
```

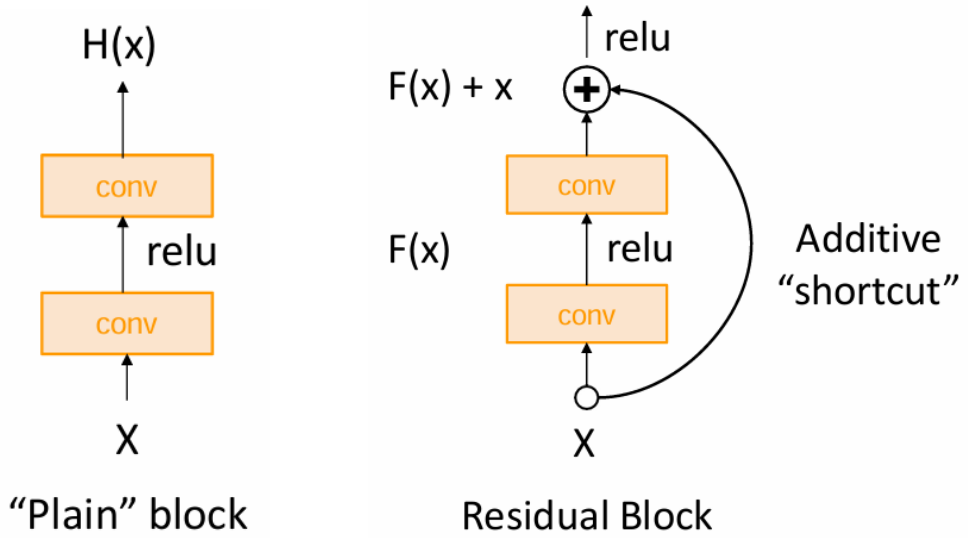
```
------------------------------------------------
📊 Class-wise IoU:
  Class 0: 0.8547
  Class 1: 0.2830
  Class 2: 0.0257
  Class 3: 0.0436
  Class 4: 0.0000
  Class 5: 0.0000
  Class 6: 0.5214
  Class 7: 0.6111
  Class 8: 0.3100
  Class 9: 0.5464
  Class 10: 0.7080
  Class 11: 0.0000
  Class 12: 0.0000
🏆 mIoU (Mean IoU): 0.3003
================================================
```

```
------------------------------------------------
📊 Class-wise IoU:
  Class 0: 0.8456
  Class 1: 0.2843
  Class 2: 0.0327
  Class 3: 0.0931
  Class 4: 0.0000
  Class 5: 0.0032
  Class 6: 0.5681
  Class 7: 0.6070
  Class 8: 0.3451
  Class 9: 0.6290
  Class 10: 0.7289
  Class 11: 0.0011
  Class 12: 0.0240
🏆 mIoU (Mean IoU): 0.3202
================================================
```

- The performance improved with fewer layers
- Problem can arise when the model is designed just to be deep

FCN-ResNet50

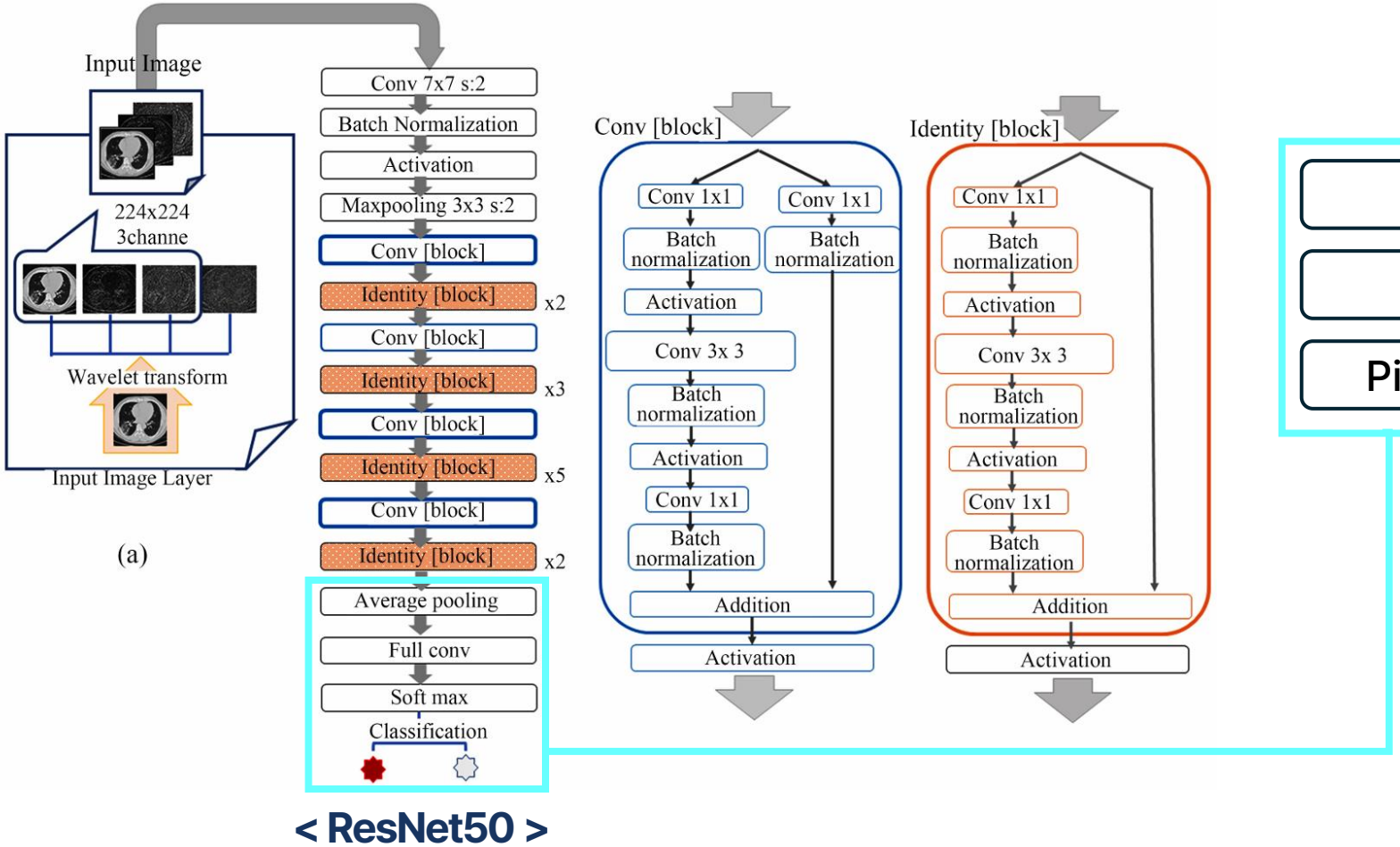
- **ResNet (Residual Networks)**
  - Deeper Model → Degradation problem (= Train Error ↑)
  - If the optimal mapping is identity (H(x) = x),
    - Traditional Networks → Learn H(x) => Hard
    - **Residual Networks** → Learn **F(x)=0** => Easy
      - ➢ Just add a small residual to "x"



He, Kaiming, et al. "Deep residual learning for image recognition." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016

FCN-ResNet50

- **Model Architecture**



< ResNet50 >

< FCN-ResNet50 >

1x1 Conv

Upsampling (Bilinear)

Pixel-wise Softmax (CrossEntropy)

- Built our model by referencing the PyTorch FCN-ResNet50 backbone.

Matsuyama, Eri. "A Deep Learning Interpretable Model for Novel Coronavirus Disease (COVID-19) Screening with Chest CT Images." Journal of Biomedical Science and Engineering, vol. 13, no. 7, 2020, pp. 140–152.

FCN-ResNet50

- **Training Process**

```
# ◆ 클래스 가중치 계산: 자주 나오는 클래스는 가중치를 낮게, 희귀 클래스는 높게
ratios = np.array([class_ratio[k] for k in sorted(class_ratio.keys())]) / 100.0
# 1/log(1.02 + p) 형태 같은 간단한 스킴
class_weights = 1.0 / np.log(1.02 + ratios)
print("\nSuggested class weights:", class_weights)
```

```
class_weights_np = run_full_eda("Training/RGB", "Training/Label")

# ◆ 클래스 가중치 정규화 (평균이 1이 되도록)
class_weights_np = class_weights_np / class_weights_np.mean()
print("Normalized class weights:", class_weights_np)
```

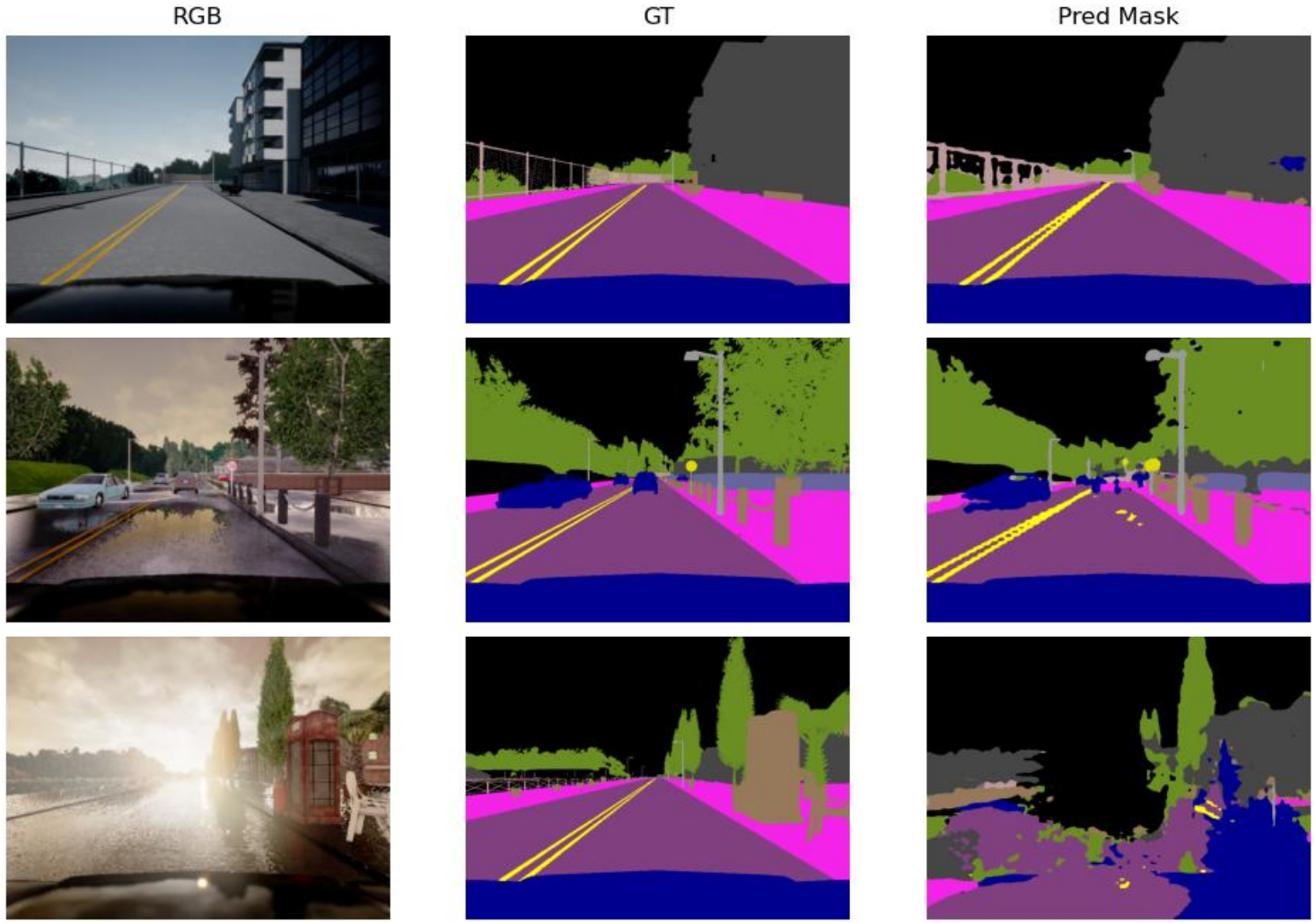
```
main_loss = F.cross_entropy(main_logits, y,
                            weight=class_weights,
                            reduction='mean')
aux_loss  = F.cross_entropy(aux_logits,  y,
                            weight=class_weights,
                            reduction='mean')
```

```
# ---------- 간단한 증강: train일 때만 ----------
if self.train:
    # 1) 랜덤 좌우 플립
    if np.random.rand() < 0.5:
        rgb = TF.hflip(rgb)
        label = np.fliplr(label).copy()

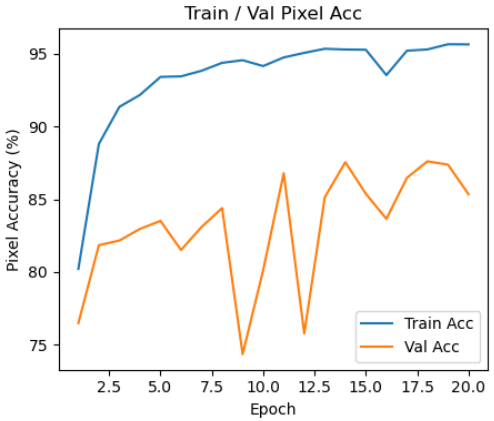
    # 2) 랜덤 크롭 (예: 512x512)
    h, w = label.shape
    crop_h, crop_w = 512, 512
    if h >= crop_h and w >= crop_w:
        top = np.random.randint(0, h - crop_h + 1)
        left = np.random.randint(0, w - crop_w + 1)
        rgb = TF.crop(rgb, top, left, crop_h, crop_w)
        label = label[top:top+crop_h, left:left+crop_w]
    # (너무 복잡하게 안 가고, 단순 예시 수준으로)
```

FCN-ResNet50

- **Training Process**








```
Pixel Accuracy: 87.61%
mIoU: 52.67%
Class  0 (None         ): IoU = 90.14%
Class  1 (Buildings    ): IoU = 73.09%
Class  2 (Fences       ): IoU = 24.49%
Class  3 (Other        ): IoU = 21.46%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 36.44%
Class  6 (RoadLines    ): IoU = 54.05%
Class  7 (Roads        ): IoU = 78.67%
Class  8 (Sidewalks    ): IoU = 69.27%
Class  9 (Vegetation   ): IoU = 70.68%
Class 10 (Vehicles     ): IoU = 83.44%
Class 11 (Walls        ): IoU = 29.59%
Class 12 (TrafficSigns): IoU = 53.43%
```

FCN-ResNet50

- **Training Process – Add Augmentation**

```python
# ---------- 데이터 증강: train일 때만 ----------
if self.train:
    # 이 이미지에 희귀 클래스가 포함되어 있는지
    is_rare_img = self.has_rare[idx]

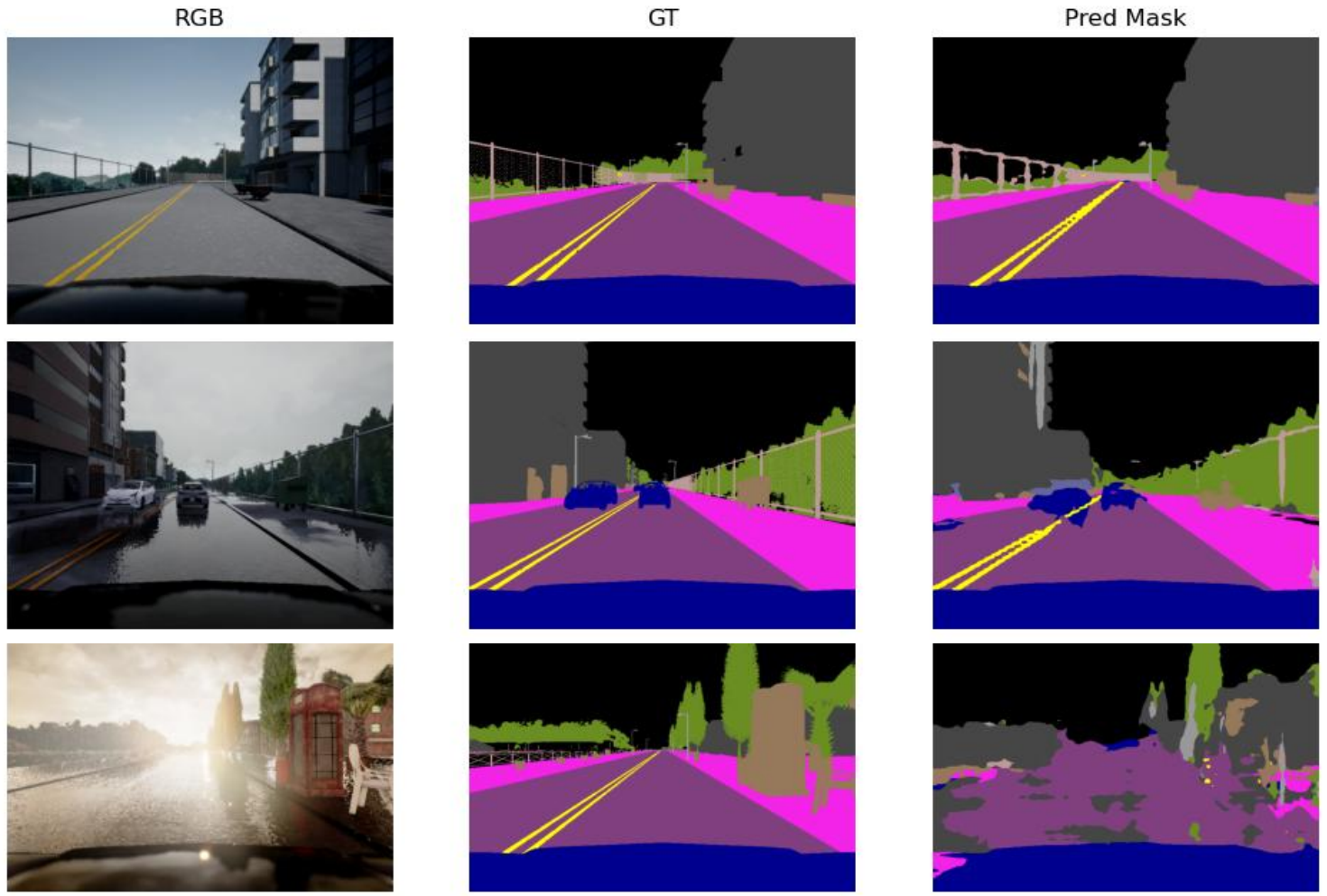
    # 희귀 이미지면 augmentation을 더 자주/강하게 적용
    # 예: 희귀 이미지 → 1.0, 일반 이미지 → 0.4
    aug_factor = 1.0 if is_rare_img else 0.4

    # (1) 랜덤 좌우 플립
    if random.random() < 0.5 * aug_factor:
        rgb = TF.hflip(rgb)
        label = np.fliplr(label).copy()

    # (2) color jitter (살짝 줄임)
    if random.random() < 0.3 * aug_factor:
        factor_b = 0.7 + 0.6 * random.random()
        factor_c = 0.7 + 0.6 * random.random()
        rgb = TF.adjust_brightness(rgb, factor_b)
        rgb = TF.adjust_contrast(rgb, factor_c)

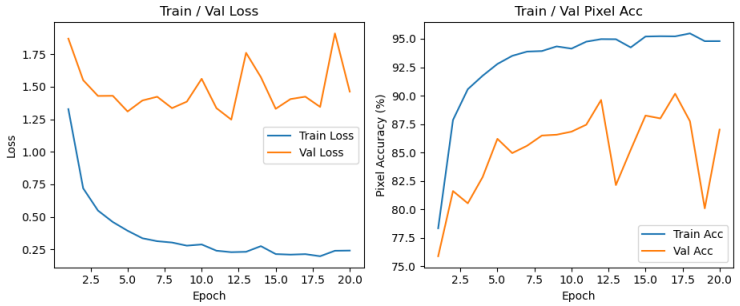
    # (3) 가우시안 블러는 희귀 이미지일 때만 조금
    if is_rare_img and random.random() < 0.15 * aug_factor:
        rgb = rgb.filter(ImageFilter.GaussianBlur(radius=1.5))

    # (4) 랜덤 크롭 (예: 512x512)
    h, w = label.shape
    crop_h, crop_w = 512, 512
    if h >= crop_h and w >= crop_w:
        top = random.randint(0, h - crop_h)
        left = random.randint(0, w - crop_w)
        rgb = TF.crop(rgb, top, left, crop_h, crop_w)
        label = label[top:top+crop_h, left:left+crop_w]
```

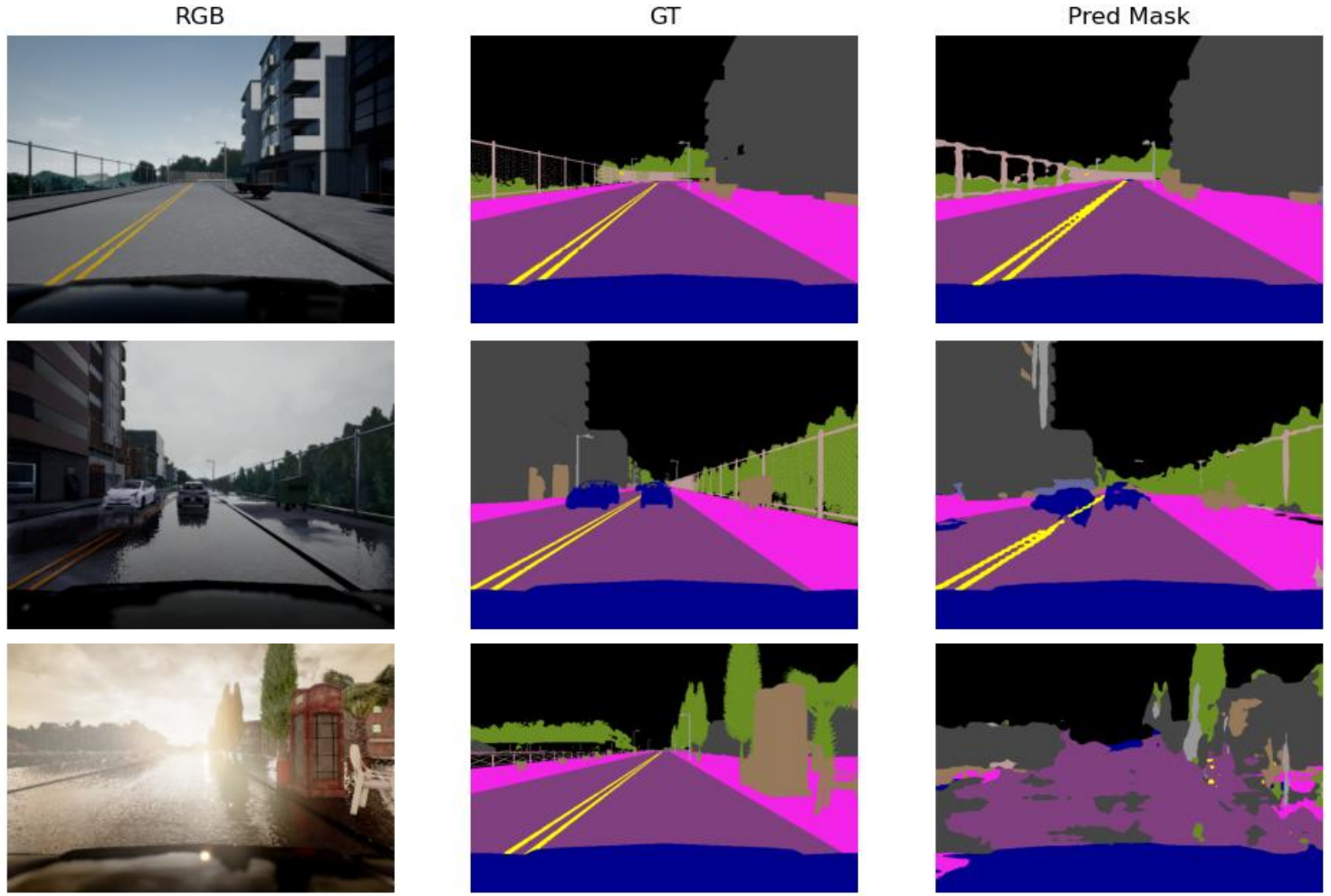


FCN-ResNet50

- **Training Process – Add Augmentation**



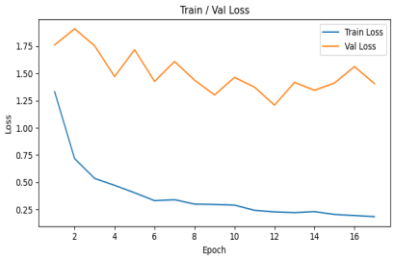

```
Pixel Accuracy: 90.17%
mIoU: 54.04%
Class  0 (None        ): IoU = 93.78%
Class  1 (Buildings   ): IoU = 66.22%
Class  2 (Fences      ): IoU = 25.19%
Class  3 (Other       ): IoU = 21.15%
Class  4 (Pedestrians ): IoU =  0.00%
Class  5 (Poles       ): IoU = 30.90%
Class  6 (RoadLines   ): IoU = 54.45%
Class  7 (Roads       ): IoU = 88.03%
Class  8 (Sidewalks   ): IoU = 71.66%
Class  9 (Vegetation  ): IoU = 72.06%
Class 10 (Vehicles    ): IoU = 95.08%
Class 11 (Walls       ): IoU = 28.73%
Class 12 (TrafficSigns): IoU = 55.30%
```



FCN-ResNet50

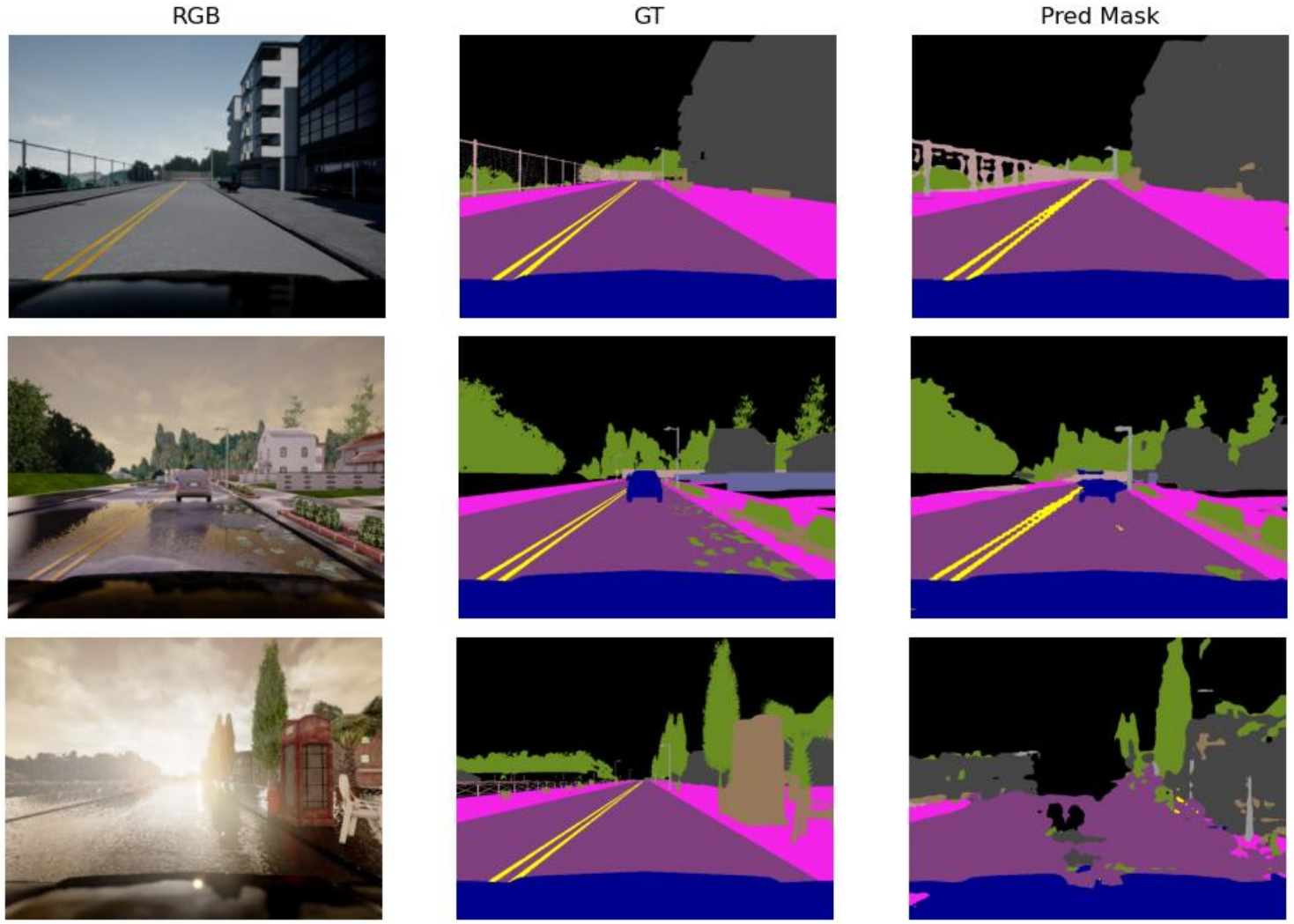
- **Training Process – Add Scheduler**

```python
# 스케쥴러 추가
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer=my_opt,
    mode='max',      # val_acc가 커질수록 좋음
    factor=0.5,      # LR = LR * 0.5
    patience=1,      # 1 epoch 동안 개선 없으면 줄임
)
```



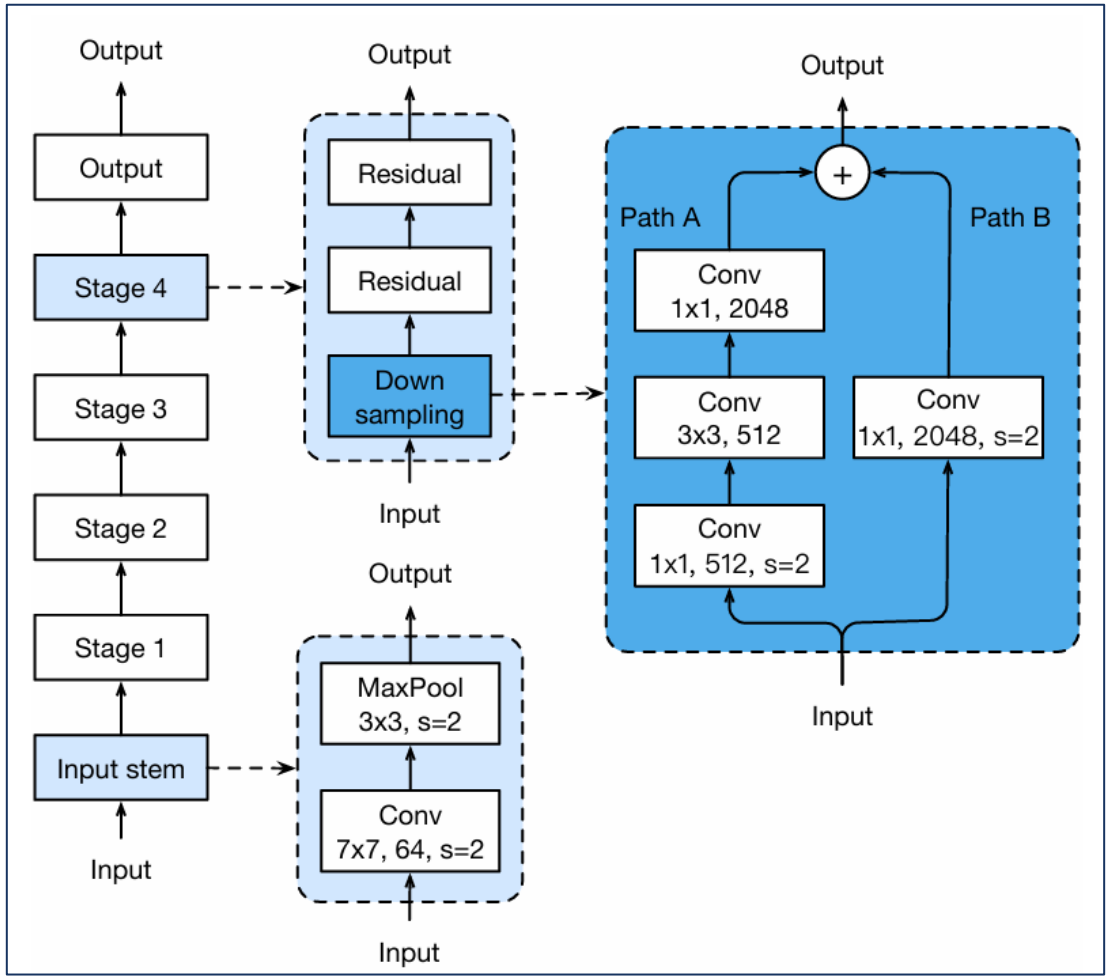



```
Pixel Accuracy: 90.16%
mIoU: 53.61%
Class  0 (None        ): IoU = 92.74%
Class  1 (Buildings   ): IoU = 71.04%
Class  2 (Fences      ): IoU = 19.00%
Class  3 (Other       ): IoU = 19.20%
Class  4 (Pedestrians ): IoU =  0.00%
Class  5 (Poles       ): IoU = 31.11%
Class  6 (RoadLines   ): IoU = 54.84%
Class  7 (Roads       ): IoU = 88.46%
Class  8 (Sidewalks   ): IoU = 70.41%
Class  9 (Vegetation  ): IoU = 73.11%
Class 10 (Vehicles    ): IoU = 94.96%
Class 11 (Walls       ): IoU = 29.58%
Class 12 (TrafficSigns): IoU = 52.45%
```

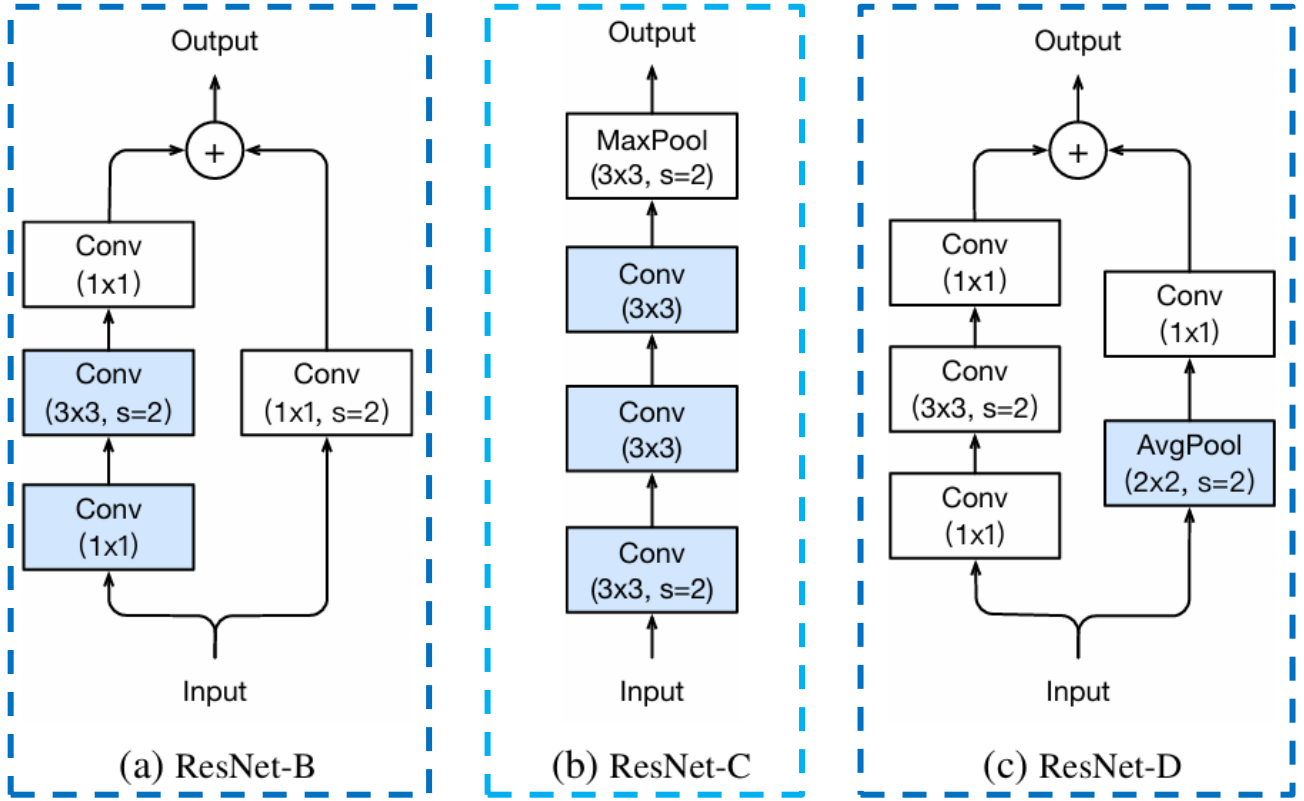


FCN-ResNet50

- **Bag of tricks**



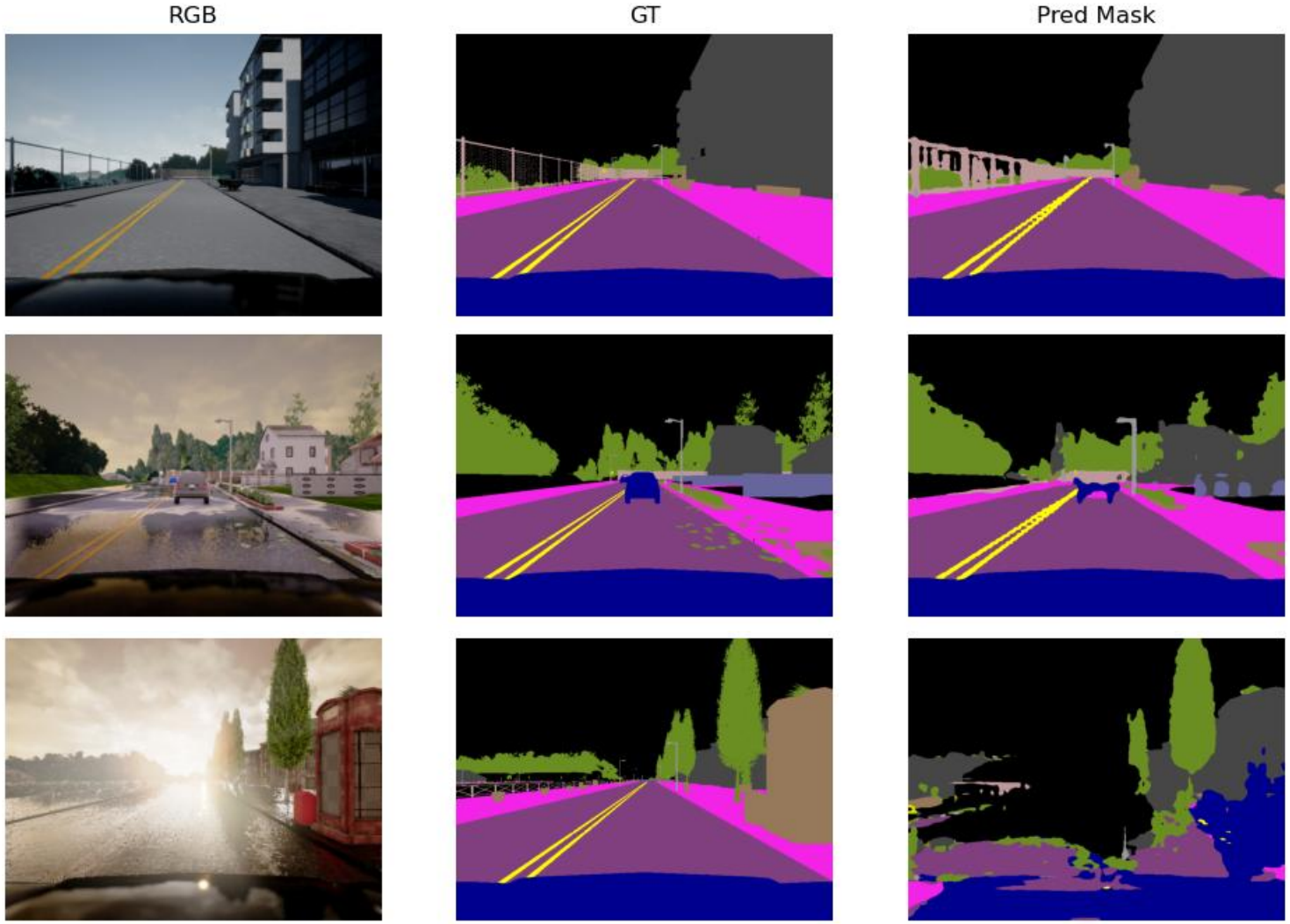
< ResNet50 >



He, Tong, et al. "Bag of tricks for image classification with convolutional neural networks." *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019.

FCN-ResNet50

- **Training Process**

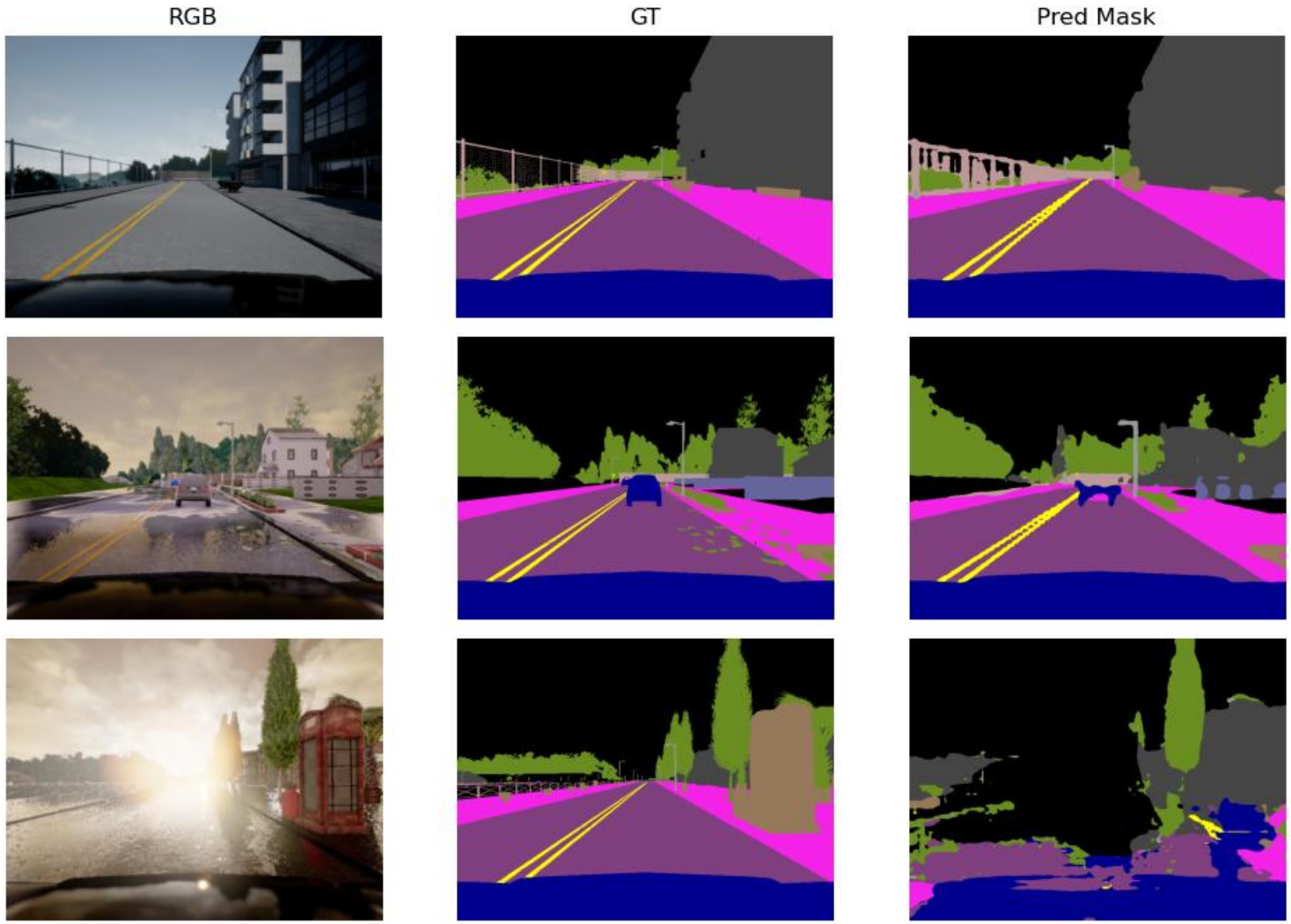


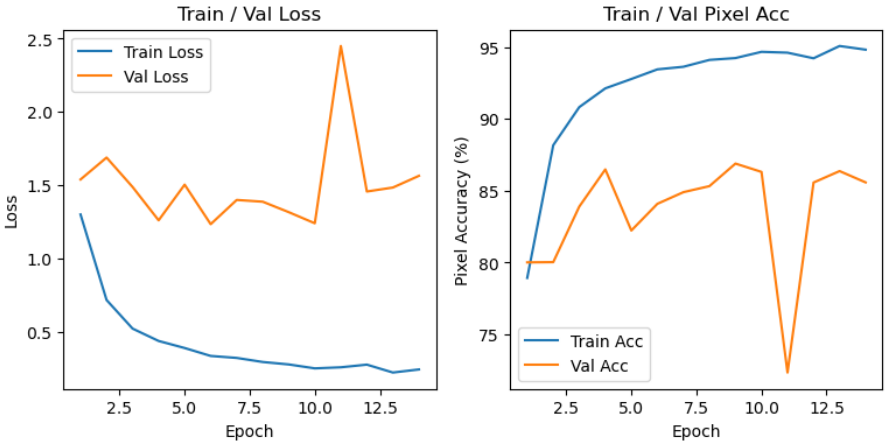



```
Pixel Accuracy: 88.02%
mIoU: 52.87%
Class  0 (None        ): IoU = 86.55%
Class  1 (Buildings   ): IoU = 75.89%
Class  2 (Fences      ): IoU = 23.43%
Class  3 (Other       ): IoU = 19.27%
Class  4 (Pedestrians ): IoU =  0.00%
Class  5 (Poles       ): IoU = 32.93%
Class  6 (RoadLines   ): IoU = 54.62%
Class  7 (Roads       ): IoU = 80.94%
Class  8 (Sidewalks   ): IoU = 70.83%
Class  9 (Vegetation  ): IoU = 69.05%
Class 10 (Vehicles    ): IoU = 92.38%
Class 11 (Walls       ): IoU = 34.15%
Class 12 (TrafficSigns): IoU = 47.31%
```

FCN-ResNet50

- **Training Process - Augmentation**

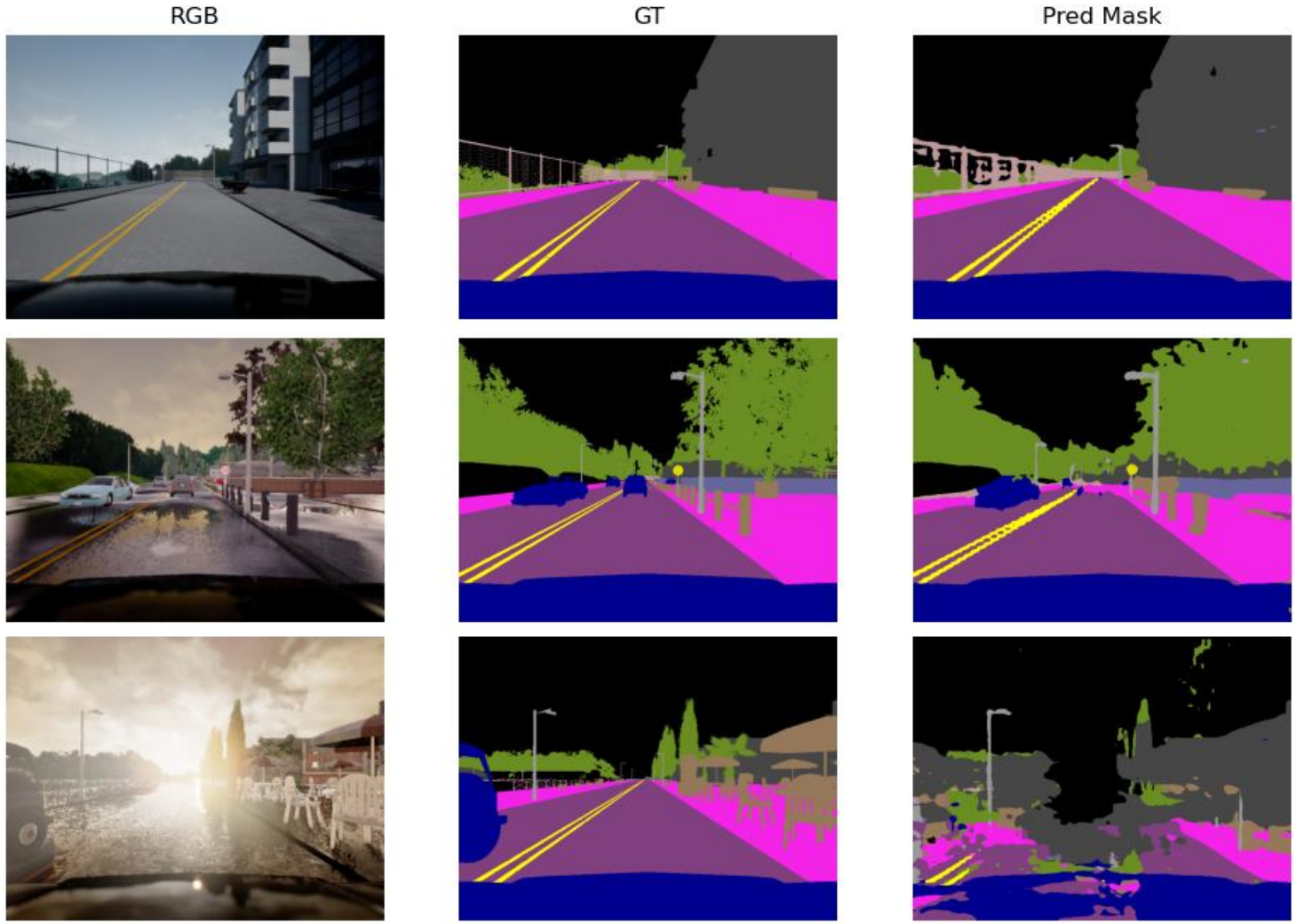


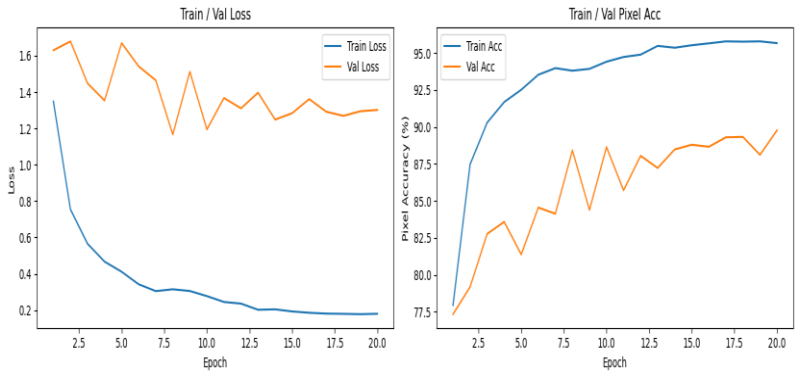



```
Pixel Accuracy: 86.90%
mIoU: 51.16%
Class  0 (None         ): IoU = 91.32%
Class  1 (Buildings    ): IoU = 56.16%
Class  2 (Fences       ): IoU = 19.15%
Class  3 (Other        ): IoU = 16.79%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 31.21%
Class  6 (RoadLines    ): IoU = 56.43%
Class  7 (Roads        ): IoU = 78.97%
Class  8 (Sidewalks    ): IoU = 68.89%
Class  9 (Vegetation   ): IoU = 69.65%
Class 10 (Vehicles     ): IoU = 90.28%
Class 11 (Walls        ): IoU = 31.78%
Class 12 (TrafficSigns): IoU = 54.47%
```

FCN-ResNet50

- **Training Process - Scheduler**






```
Pixel Accuracy: 89.78%
mIoU: 55.11%
Class  0 (None         ): IoU = 92.15%
Class  1 (Buildings    ): IoU = 67.49%
Class  2 (Fences       ): IoU = 23.78%
Class  3 (Other        ): IoU = 20.65%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 38.70%
Class  6 (RoadLines    ): IoU = 57.28%
Class  7 (Roads        ): IoU = 86.14%
Class  8 (Sidewalks    ): IoU = 75.52%
Class  9 (Vegetation   ): IoU = 71.35%
Class 10 (Vehicles     ): IoU = 94.20%
Class 11 (Walls        ): IoU = 36.62%
Class 12 (TrafficSigns): IoU = 52.55%
```

FCN-ResNet50

- **Performance Comparison**

```
Pixel Accuracy: 87.61%
mIoU: 52.67%
Class  0 (None         ): IoU = 90.14%
Class  1 (Buildings    ): IoU = 73.09%
Class  2 (Fences       ): IoU = 24.49%
Class  3 (Other        ): IoU = 21.46%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 36.44%
Class  6 (RoadLines    ): IoU = 54.05%
Class  7 (Roads        ): IoU = 78.67%
Class  8 (Sidewalks    ): IoU = 69.27%
Class  9 (Vegetation   ): IoU = 70.68%
Class 10 (Vehicles     ): IoU = 83.44%
Class 11 (Walls        ): IoU = 29.59%
Class 12 (TrafficSigns): IoU = 53.43%
```

```
Pixel Accuracy: 90.17%
mIoU: 54.04%
Class  0 (None         ): IoU = 93.78%
Class  1 (Buildings    ): IoU = 66.22%
Class  2 (Fences       ): IoU = 25.19%
Class  3 (Other        ): IoU = 21.15%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 30.90%
Class  6 (RoadLines    ): IoU = 54.45%
Class  7 (Roads        ): IoU = 88.03%
Class  8 (Sidewalks    ): IoU = 71.66%
Class  9 (Vegetation   ): IoU = 72.06%
Class 10 (Vehicles     ): IoU = 95.08%
Class 11 (Walls        ): IoU = 28.73%
Class 12 (TrafficSigns): IoU = 55.30%
```

```
Pixel Accuracy: 90.16%
mIoU: 53.61%
Class  0 (None         ): IoU = 92.74%
Class  1 (Buildings    ): IoU = 71.04%
Class  2 (Fences       ): IoU = 19.00%
Class  3 (Other        ): IoU = 19.20%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 31.11%
Class  6 (RoadLines    ): IoU = 54.84%
Class  7 (Roads        ): IoU = 88.46%
Class  8 (Sidewalks    ): IoU = 70.41%
Class  9 (Vegetation   ): IoU = 73.11%
Class 10 (Vehicles     ): IoU = 94.96%
Class 11 (Walls        ): IoU = 29.58%
Class 12 (TrafficSigns): IoU = 52.45%
```

```
Pixel Accuracy: 88.02%
mIoU: 52.87%
Class  0 (None         ): IoU = 86.55%
Class  1 (Buildings    ): IoU = 75.89%
Class  2 (Fences       ): IoU = 23.43%
Class  3 (Other        ): IoU = 19.27%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 32.93%
Class  6 (RoadLines    ): IoU = 54.62%
Class  7 (Roads        ): IoU = 80.94%
Class  8 (Sidewalks    ): IoU = 70.83%
Class  9 (Vegetation   ): IoU = 69.05%
Class 10 (Vehicles     ): IoU = 92.38%
Class 11 (Walls        ): IoU = 34.15%
Class 12 (TrafficSigns): IoU = 47.31%
```

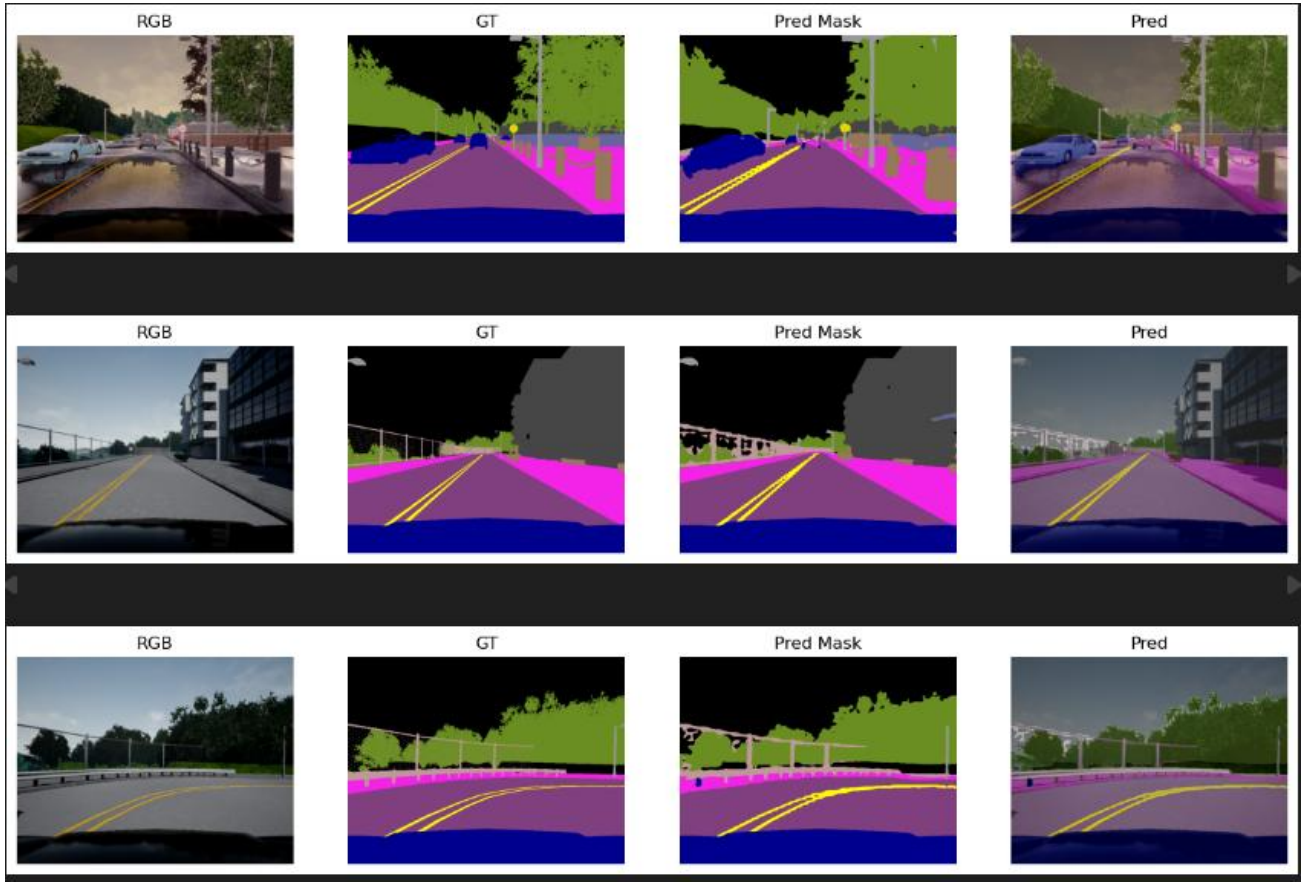
```
Pixel Accuracy: 86.90%
mIoU: 51.16%
Class  0 (None         ): IoU = 91.32%
Class  1 (Buildings    ): IoU = 56.16%
Class  2 (Fences       ): IoU = 19.15%
Class  3 (Other        ): IoU = 16.79%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 31.21%
Class  6 (RoadLines    ): IoU = 56.43%
Class  7 (Roads        ): IoU = 78.97%
Class  8 (Sidewalks    ): IoU = 68.89%
Class  9 (Vegetation   ): IoU = 69.65%
Class 10 (Vehicles     ): IoU = 90.28%
Class 11 (Walls        ): IoU = 31.78%
Class 12 (TrafficSigns): IoU = 54.47%
```

```
Pixel Accuracy: 89.78%
mIoU: 55.11%
Class  0 (None         ): IoU = 92.15%
Class  1 (Buildings    ): IoU = 67.49%
Class  2 (Fences       ): IoU = 23.78%
Class  3 (Other        ): IoU = 20.65%
Class  4 (Pedestrians  ): IoU =  0.00%
Class  5 (Poles        ): IoU = 38.70%
Class  6 (RoadLines    ): IoU = 57.28%
Class  7 (Roads        ): IoU = 86.14%
Class  8 (Sidewalks    ): IoU = 75.52%
Class  9 (Vegetation   ): IoU = 71.35%
Class 10 (Vehicles     ): IoU = 94.20%
Class 11 (Walls        ): IoU = 36.62%
Class 12 (TrafficSigns): IoU = 52.55%
```

Basic	Best acc	96.60%	97.07%	96.35%
	Middle acc	92.70%	92.54%	92.03%
	Worst acc	55.28%	71.78%	74.88%
Bag of tricks	Best acc	96.17%	96.17%	96.75%
	Middle acc	92.11%	92.11%	93.09%
	Worst acc	62.45%	62.45%	63.75%

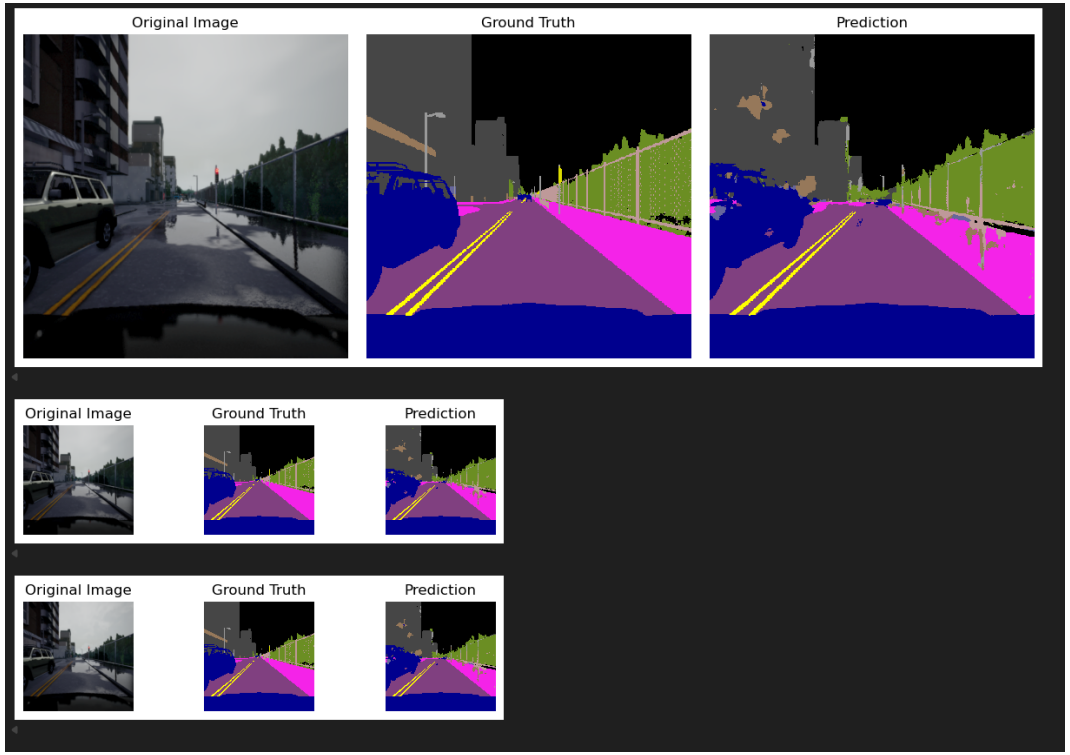
FCN-ResNet50

+ Augmentation applied, focusing on minority classes+ Scheduling



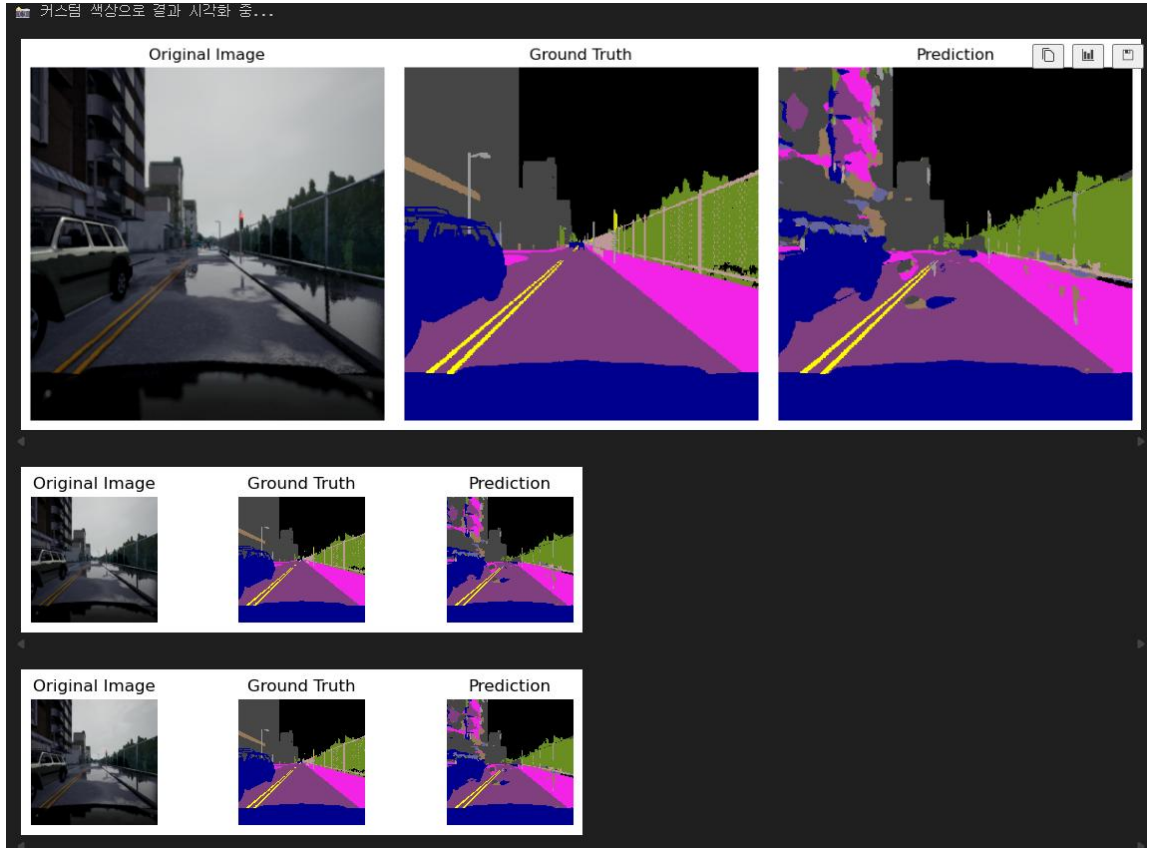

```
Pixel Accuracy: 89.78%
mIoU: 55.11%
Class  0 (None        ): IoU = 92.15%
Class  1 (Buildings   ): IoU = 67.49%
Class  2 (Fences      ): IoU = 23.78%
Class  3 (Other       ): IoU = 20.65%
Class  4 (Pedestrians ): IoU =  0.00%
Class  5 (Poles       ): IoU = 38.70%
Class  6 (RoadLines   ): IoU = 57.28%
Class  7 (Roads       ): IoU = 86.14%
Class  8 (Sidewalks   ): IoU = 75.52%
Class  9 (Vegetation  ): IoU = 71.35%
Class 10 (Vehicles    ): IoU = 94.20%
Class 11 (Walls       ): IoU = 36.62%
Class 12 (TrafficSigns): IoU = 52.55%
```

Basic UNet




```
[Epoch 10/10]
Val Loss: 0.4569
--------------------------------------------------
  📊 Class-wise IoU:
    Class 0: 0.8966
    Class 1: 0.6676
    Class 2: 0.2480
    Class 3: 0.1692
    Class 4: 0.0000
    Class 5: 0.2291
    Class 6: 0.6772
    Class 7: 0.8566
    Class 8: 0.7196
    Class 9: 0.7317
    Class 10: 0.9113
    Class 11: 0.2132
    Class 12: 0.0098
  🏆 mIoU (Mean IoU): 0.4869
==================================================
🎉 모든 학습이 완료되었습니다!
```

UNet++




```
[Epoch 10/10]
Val Loss: 0.6046
------------------------------------------------
  📊 Class-wise IoU:
    Class 0: 0.8907
    Class 1: 0.4816
    Class 2: 0.1755
    Class 3: 0.1336
    Class 4: 0.0000
    Class 5: 0.2341
    Class 6: 0.6641
    Class 7: 0.7731
    Class 8: 0.6793
    Class 9: 0.7318
    Class 10: 0.8518
    Class 11: 0.1990
    Class 12: 0.0000
  🏆 mIoU (Mean IoU): 0.4473
================================================
🎉 모든 학습이 완료되었습니다!
```

5. Final Model Selection

CONTENTS

01 2D Sematic Segmentation

02 Model Introduction
FCN, Unet

03 EDA

04 Model Selection Process

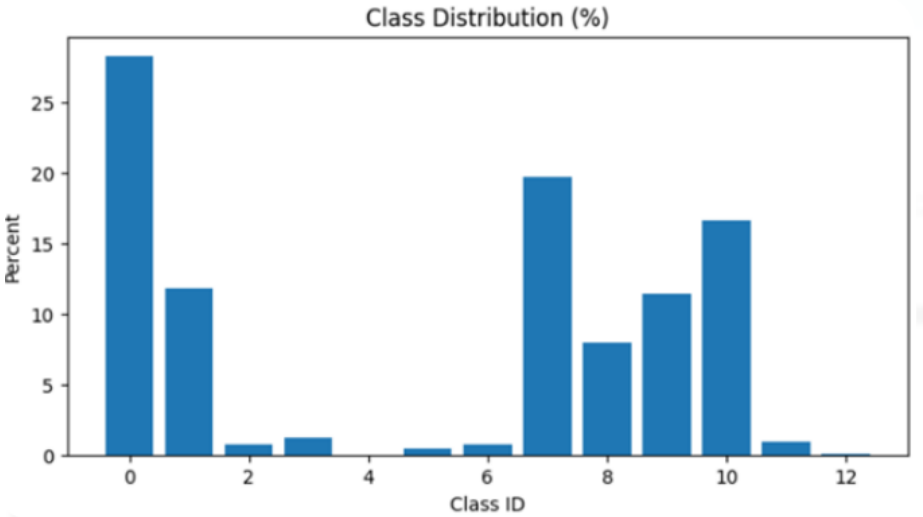
05 Final Model Selection

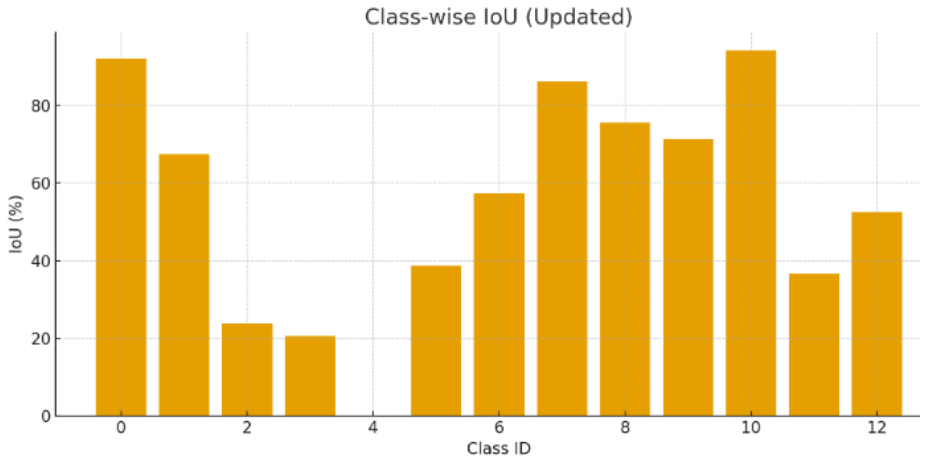
06 Limitations and Improvement Measures

Final Model Selection- FCN-ResNet50

- **Evaluation Metrics**
  - : IOU/MIOU → MIOU 55.11 % (59.70% excluding pedestrians)
  - Pixel Accuracy : 89.78%

Value	Tag	IOU(%)	Pixel ratio(%)
0	None	92.15%	28.237%
1	Buildings	67.49%	11.804%
2	Fences	23.78%	0.781%
3	Other	20.65%	1.202%
4	Pedestrians	0.00%	0.006%
5	Poles	38.70%	0.490%
6	RoldLines	57.28%	0.744%
7	Roads	86.14%	19.557%
8	Sidewalks	75.52%	7.973%
9	Vegetation	71.35%	11.463%
10	Vehicles	94.20%	16.661%
11	Walks	36.62%	0.893%
12	TrafficSigns	52.55%	0.081%





6. Limitations and Improvement Measures

CONTENTS

01 2D Sematic Segmentation

02 Model Introduction
FCN, Unet

03 EDA

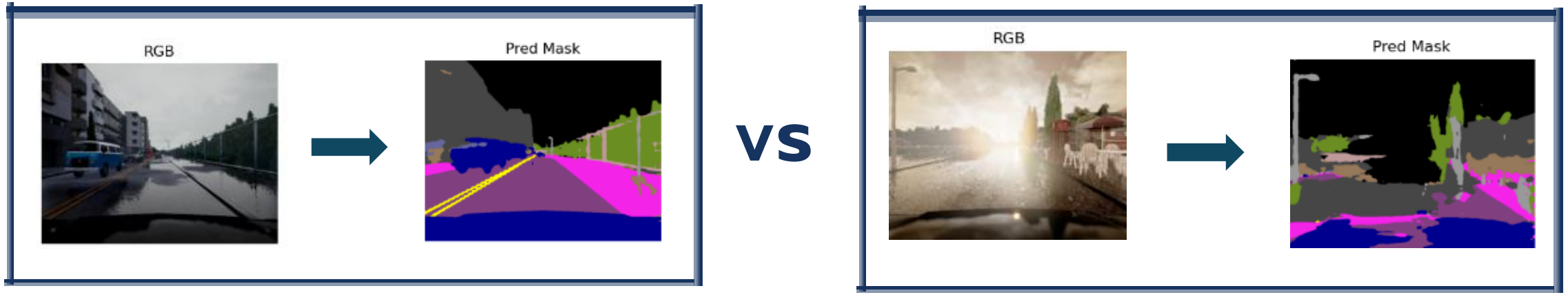
04 Model Selection Process

05 Final Model Selection

06 Limitations and Improvement Measures

Limitations

- Challenges in accurately modeling illumination variations



- Difficulty predicting rare classes, especially pedestrians

Value	Tag	IOU(%)	Pixel ratio(%)
2	Fences	25.19%	0.781%
3	Other	21.15%	1.202%
4	Pedestrians	0.00%	0.006%
11	Walks	28.73%	0.893%

Improvement Measures

- Challenges in accurately modeling illumination variations

➡ Train with paired noisy and clear images and apply augmentation to handle illumination variations

- Difficulty predicting rare classes, especially pedestrians

➡ Solving the problem using a Dual-Resolution Coarse-to-Superpixel Framework

Chen, G., Liu, R., Li, K., Li, P., Lin, M., & Chen, J. (2021). Adaptive multi-scale feature fusion for semantic segmentation. Multimedia Systems

Reference

- https://wikidocs.net/120209
- https://geunuk.tistory.com/448
- https://pasus.tistory.com/204
- https://www.mdpi.com/1999-5903/15/6/205
- Long, J., Shelhamer, E., & Darrell, T. (2015). Fully Convolutional Networks for Semantic Segmentation. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- Chen, G., Liu, R., Li, K., Li, P., Lin, M., & Chen, J. (2021). Adaptive multi-scale feature fusion for semantic segmentation. Multimedia Systems
- https://theaisummer.com/unet-architectures/
- https://www.jeremyjordan.me/semantic-segmentation/
- https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/
- He, Kaiming, et al. "Deep residual learning for image recognition." Proceedings of the IEEE conference on computer vision and pattern recognition. 2016
- Automotive Artificial Intelligence Lecture 12. CNN Part2
- Matsuyama, Eri. “A Deep Learning Interpretable Model for Novel Coronavirus Disease (COVID-19) Screening with Chest CT Images.” Journal of Biomedical Science and Engineering, vol. 13, no. 7, 2020, pp. 140–152.
- He, Tong, et al. "Bag of tricks for image classification with convolutional neural networks." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019.
- Chen, G., Liu, R., Li, K., Li, P., Lin, M., & Chen, J. (2021). Adaptive multi-scale feature fusion for semantic segmentation. Multimedia Systems